

Corso di Laurea in Ingegneria e Scienze Informatiche

**Generazione automatica di dataset
text-to-SPARQL per fine-tuning di
LLM su knowledge graph
clinico-oncologico**

Tesi di laurea in:
PROGRAMMAZIONE

Relatore

Prof.ssa Antonella Carbonaro

Candidato

Matteo Tonelli

Sommario

Il lavoro proposto si colloca nel progetto Cancer Virtual Lab (CVL), il quale raccoglie i dati clinici oncologici dell'IRCCS IRST "Dino Amadori" in un Knowledge Graph interrogabile tramite SPARQL. La natura formale di SPARQL rappresenta però una barriera per il personale sanitario e un'interfaccia in linguaggio naturale (Text-to-SPARQL) permetterebbe di abbattere tale barriera, ma richiederebbe un modello linguistico addestrato sull'ontologia specifica di CVL, per la quale non esistono dataset di addestramento pubblici.

Questo lavoro progetta e implementa una pipeline automatizzata a due stadi per la generazione di un dataset Text-to-SPARQL sintetico, garantendo correttezza e varietà. Il primo stadio esplora lo schema del grafo e campiona combinazioni di vincoli sintatticamente corrette, coprendo diverse entità organizzate in gruppi di priorità basati sulla centralità clinica. Il secondo stadio applica un meccanismo di mascheramento dei valori letterali e un ciclo iterativo di parafrasi guidata per generare varianti in più registri stilistici distinti, aumentando la ricchezza linguistica senza alterare la semantica delle query.

La pipeline ha prodotto in fase di testing un dataset di 14.021 esempi, strutturato con una rigorosa separazione per record sorgente tra gli insiemi di addestramento, validazione e test per evitare data leakage. Un modello Gemma 3 270M, sottoposto a fine-tuning mediante LoRA su questo dataset, ha raggiunto un exact match del 46% sull'insieme di test in-distribution, individuando il checkpoint ottimale tramite validation loss e superando nettamente il full fine-tuning (18%). L'analisi degli errori residui ha mostrato un tasso dello 0% di query sintatticamente non valide, dimostrando che il modello apprende saldamente la grammatica SPARQL. Vengono infine analizzati i problemi di qualità emersi nel dataset, delineando le relative strategie di risoluzione.

Indice

Sommario	iii
1 Introduzione	1
2 Background	3
2.1 Cancer Virtual Lab	4
2.1.1 La Sfida della Frammentazione dei Dati Clinici	5
2.1.2 Standard di Interoperabilità: da HL7 FHIR a RDF	5
2.1.3 L'Assistente Virtuale e la Necessità di Dataset Dedicati	6
2.2 Knowledge Graph, RDF e Strumenti SPARQL	7
2.2.1 Dai Dati Strutturati ai Knowledge Graph	8
2.2.2 Resource Description Framework (RDF)	8
2.2.3 SPARQL	9
2.2.4 Validazione Sintattica Determinista con RDFLib	9
2.3 Architettura Software e Pattern	10
2.3.1 Il Pattern Architetturale Mixin	10
2.3.2 Prototipazione e Interfacce Reattive con Streamlit	11
2.4 Lavori Correlati e Benchmark di Text-to-SPARQL	12
2.4.1 Approcci Basati su Template Rigidi e Corpora Sintetici	12
2.4.2 Sintesi Guidata da Regole e Grammatiche Formali (MK-SQuIT)	12
2.4.3 Generazione Indotta da Large Language Model	13
2.4.4 Motivazioni della Scelta Metodologica nel Contesto CVL	14
2.5 Fine-tuning di Large Language Model	14
2.5.1 Adattamento supervisionato (SFT)	14
2.5.2 Full Fine-Tuning	15
2.5.3 Tecniche di adattamento efficiente (PEFT, LoRA e QLoRA)	15
2.5.4 Funzione di perdita e formato delle istruzioni	16

3	Analisi	17
3.1	Obiettivi	17
3.2	Vincoli	18
3.3	Requisiti funzionali	19
3.4	Requisiti non funzionali	19
4	Progettazione	21
4.1	Analisi dei problemi	21
4.2	Design del generatore	22
4.2.1	La pipeline di generazione	22
4.2.2	Modello del dominio	23
4.2.3	Sistema dei parametri	25
4.2.4	Meccanismo di estensione	27
4.3	Design dell'augmentation	27
4.3.1	Mascheramento dei valori letterali	28
4.3.2	Generazione iterativa e validazione	29
4.3.3	Varietà stilistica	30
5	Implementazione	31
5.1	Implementazione del generatore	31
5.1.1	Architettura a mixin	31
5.1.2	Scoperta della mappa del grafo	34
5.1.3	Costruzione delle route	35
5.1.4	Pianificazione: stima combinatoria e campionamento	36
5.1.5	Esecuzione sequenziale e parallela	38
5.1.6	Costruzione dello scheletro: percorso online e offline	40
5.1.7	Riempimento dei valori	43
5.1.8	Generazione del prompt e tracciamento degli span	43
5.1.9	Controlli di qualità e deduplicazione	45
5.1.10	Generazione manuale a entità singola	46
5.1.11	Configurazione dei tier da file esterno	47
5.1.12	Split del dataset	48
5.2	Implementazione dell'augmentation	49
5.2.1	Mascheramento	50
5.2.2	Riformulazione booleana e suggerimenti naturali	51
5.2.3	Suddivisione in chunk per record complessi	52
5.2.4	Generazione iterativa: stile, retry e fallback	53
5.2.5	Validazione: due funzioni per due modalità	54
5.2.6	Rotazione degli stili e memoria anti-ripetizione	55
5.2.7	Statistiche di qualità e resilienza	56
5.2.8	La modalità senza mascheramento	56

6	Valutazione	59
6.1	Testing	59
6.2	Experimental evaluation / demos	64
7	Conclusioni e sviluppi futuri	69
7.1	Sviluppi futuri	70
7.1.1	Risoluzione dei problemi di qualità	70
7.1.2	Altre direzioni di sviluppo	71
	Bibliografia	73

Elenco delle figure

4.1	Modello di dominio semplificato: Patient come nodo centrale, con una classe rappresentativa per ciascuno dei sei gruppi di priorità . . .	25
4.2	Modello esemplificativo dei parametri filtrabili	26
6.1	Occorrenze riscontrate per ciascuno dei quattro problemi di qualità quantificabili (il terzo problema, individuato tramite lettura diretta anziché conteggio sistematico, non compare in figura).	61
6.2	Distribuzione delle varianti tra i dieci stili linguistici, nell'insieme di addestramento.	64
6.3	Andamento della train e della validation loss durante l'addestramento di 15 000 iterazioni. Il checkpoint scelto per la valutazione finale, in corrispondenza del minimo della validation loss, è evidenziato dalla linea tratteggiata.	66
6.4	Confronto dell'exact match tra le configurazioni testate, sia sull'insieme di test in stile coerente con l'addestramento sia, dove disponibile, sull'insieme fuori distribuzione.	67
6.5	Distribuzione delle cinquantaquattro righe non corrispondenti per categoria di errore, sul checkpoint scelto (iterazione 7 000).	68

Elenco dei listati

5.1	Composizione della classe principale del generatore tramite mixin. .	31
5.2	Controllo dell'interruzione durante la generazione.	33
5.3	Costanti di classe utilizzate dal generatore.	33
5.4	Struttura semplificata della mappa del grafo, ridotta a titolo esemplificativo a una sola proprietà e una sola relazione dell'entità Patient.	34
5.5	Visita in profondità della mappa del grafo per la costruzione delle route disponibili a partire da un'entità radice.	35
5.6	Campionamento casuale di un insieme di route indipendenti tra loro per un singolo record.	38
5.7	Scelta tra esecuzione sequenziale e parallela in base al numero di worker configurati.	39
5.8	Configurazione condivisa usata nei run di produzione, per entrambe le liste di tier.	40
5.9	Lo scheletro appena ricostruito per i terminal gender e birth date , ancora privo di valori.	42
5.10	Riempimento di un nodo dello scheletro con valori concreti, distinguendo tra intervalli e insiemi discreti.	43
5.11	Lo stesso scheletro di listato 5.9, dopo il riempimento: elenco discreto per il campo categoriale, intervallo per quello di tipo data. .	43
5.12	Costruzione del prompt segmento per segmento, con tracciamento diretto della posizione di ciascun valore letterale.	44
5.13	Il primo controllo: verifica che i vincoli scelti siano effettivamente presenti nella query SPARQL restituita.	45
5.14	Il secondo controllo: il parsing sintattico tramite rdflib è condizionato dal parametro di configurazione.	46
5.15	Suddivisione del dataset in train, validation e test, stratificata per entità di origine.	48
5.16	Generazione dei placeholder e mascheramento del testo a partire dagli span di valore.	50
5.17	Eliminazione anticipata dei placeholder booleani, sostituiti con una forma proposizionale.	51

5.18	Suddivisione del testo mascherato in chunk, rispettando il vincolo sugli intervalli <i>between...and....</i>	52
5.19	Ciclo di generazione e validazione per un singolo chunk, con fallback esplicito in caso di fallimento persistente.	53
5.20	La funzione di validazione condivisa da chunk e varianti complete, con soglia di somiglianza adattiva.	54
5.21	Assegnazione ciclica degli stili e memoria a finestra delle generazioni recenti.	55
5.22	Calcolo della resa complessiva di un run, a partire dagli slot totali e da quelli completamente falliti.	56
5.23	La modalità senza mascheramento, mantenuta come termine di paragone esplicito e non come percorso di produzione alternativo. . . .	56

Capitolo 1

Introduzione

Il Cancer Virtual Lab (CVL) è una piattaforma che raccoglie i dati clinici oncologici dell'IRST "Dino Amadori" in un knowledge graph interrogabile tramite SPARQL, il linguaggio di interrogazione standard per i dati strutturati secondo il modello RDF (capitolo 2). SPARQL è un linguaggio formale, e la sua sintassi richiede una competenza tecnica che il personale clinico e i ricercatori del dominio oncologico non possiedono tipicamente, creando una barriera tra la ricchezza dei dati disponibili e la possibilità concreta di interrogarli senza il supporto di uno specialista informatico. Un'interfaccia in linguaggio naturale capace di tradurre una domanda posta in italiano o in inglese nella query SPARQL corrispondente ridurrebbe questa barriera, ma richiede un modello linguistico specificamente addestrato a questo compito: un modello generico, per quanto capace su compiti generali, non conosce la struttura specifica dell'ontologia di CVL, i nomi delle sue classi e proprietà, né le convenzioni con cui le query sono tipicamente costruite in questo contesto.

Addestrare un simile modello richiede a sua volta un dataset di addestramento composto da coppie di domande in linguaggio naturale e query SPARQL corrispondenti, specifiche per l'ontologia di CVL. Un dataset di questo tipo non esiste in partenza, e costruirlo manualmente, coppia per coppia, non è una strada praticabile alla scala necessaria per un addestramento efficace. Questo lavoro affronta questo problema progettando e implementando una pipeline automatica in grado di generare un simile dataset, e ne verifica l'efficacia addestrando concretamente

un modello linguistico sui dati prodotti.

La pipeline sviluppata si articola in due stadi. Il primo genera in modo strutturato coppie di prompt e query SPARQL valide, esplorando sistematicamente lo schema del knowledge graph e campionando combinazioni di vincoli sintatticamente corrette. Il secondo arricchisce ciascuna coppia con più formulazioni linguistiche alternative della stessa domanda, aumentando la varietà stilistica del dataset senza alterarne la correttezza semantica. Il modello linguistico risultante dal fine-tuning su questo dataset è stato infine valutato sperimentalmente, confrontando diverse configurazioni di addestramento e analizzando la natura degli errori residui.

Struttura della tesi

Il capitolo 2 introduce il contesto tecnico necessario a seguire il resto del lavoro: la piattaforma CVL, i concetti di knowledge graph, RDF e SPARQL, i lavori correlati sulla generazione di dataset text-to-SPARQL, e le tecniche di fine-tuning di modelli linguistici impiegate più avanti. Il capitolo 3 definisce gli obiettivi, i vincoli e i requisiti del sistema. Il capitolo 4 descrive le decisioni progettuali alla base sia del generatore sia dello stadio di augmentation. Il capitolo 5 ne documenta l'implementazione concreta. Il capitolo 6 riporta i risultati sperimentali, sia sul dataset prodotto sia sul modello linguistico addestrato a partire da esso. Il capitolo 7 chiude il lavoro con una sintesi dei risultati ottenuti e le direzioni di sviluppo future.

Capitolo 2

Background

Questo capitolo fornisce il quadro teorico, metodologico e tecnologico necessario a comprendere il contesto applicativo e le decisioni progettuali alla base dell'intero lavoro di tesi. La generazione automatica di dataset sintetici per la traduzione da linguaggio naturale a linguaggi formali di interrogazione (*Text-to-SPARQL*) si colloca infatti all'intersezione tra diverse discipline informatiche: l'ingegneria del Web Semantico, la modellazione di dati clinici e il *Natural Language Processing* (NLP) applicato ai modelli linguistici di grandi dimensioni (*Large Language Model*, LLM).

Per fornire una visione più strutturata, il capitolo è organizzato in cinque sezioni interconnesse:

- **Cancer Virtual Lab (sezione 2.1):** descrive la piattaforma del Cancer Virtual Lab (CVL), illustrando le sfide legate alla frammentazione dei dati oncologici, l'adozione dello standard internazionale HL7 FHIR e le motivazioni cliniche che guidano la realizzazione di un'interfaccia di interrogazione in linguaggio naturale.
- **Knowledge Graph, RDF e Strumenti SPARQL (sezione 2.2):** introduce le nozioni fondamentali sui Knowledge Graph e sul modello di rappresentazione a triple RDF (*Resource Description Framework*). Viene inoltre approfondito il linguaggio di interrogazione SPARQL, insieme al ruolo degli strumenti software locali (in particolare la libreria Python `rdflib`) utilizzati per la validazione sintattica deterministica delle query.

- **Architettura Software e Pattern (sezione 2.3):** analizza le scelte tecnologiche e i pattern di progettazione software adottati nell'ingegneria del sistema, tra cui l'architettura a *Mixin* per la composizione modulare delle responsabilità e il framework **Streamlit** per la prototipazione di interfacce interattive di monitoraggio e controllo.
- **Lavori Correlati e Benchmark di Text-to-SPARQL (sezione 2.4):** presenta una rassegna dello stato dell'arte sui dataset di riferimento per il compito di traduzione Text-to-SPARQL, evidenziando le limitazioni dei benchmark generali (come LC-QuAD) quando applicati a domini specialistici e analizzando le criticità legate all'uso diretto di modelli generativi senza meccanismi di contenimento delle allucinazioni.
- **Fine-Tuning di Modelli Linguistici nel Dominio Clinico (sezione 2.5):** approfondisce le tecniche di adattamento supervisionato (SFT), ponendo a confronto il *Full Fine-Tuning* con le strategie di addestramento efficiente *Parameter-Efficient Fine-Tuning* (PEFT) come LoRA e QLoRA. La sezione motiva inoltre la scelta di modelli ultra-compatti per l'esecuzione locale su dati sensibili e descrive l'adozione del mascheramento della loss sui token di input (*Completion-Only Data Collator*).

2.1 Cancer Virtual Lab

Il lavoro descritto in questa tesi si inserisce direttamente nel contesto applicativo e tecnologico del progetto *Cancer Virtual Lab* (CVL) [AAG⁺25], una piattaforma avanzata di *AI Semantica* progettata per supportare la ricerca oncologica e la pratica clinica guidata dai dati. Il progetto è frutto di una collaborazione multidisciplinare tra enti di ricerca clinica e tecnologica, tra cui l'IRCCS Istituto Romagnolo per lo Studio dei Tumori (IRST) "Dino Amadori" di Meldola e l'Università di Bologna.

2.1.1 La Sfida della Frammentazione dei Dati Clinici

Nel contesto della pratica oncologica reale, le informazioni cliniche e diagnostiche dei pazienti risultano intrinsecamente eterogenee, complesse e frammentate. I dati sanitari si trovano tipicamente segregati all'interno di silos informativi autonomi e non comunicanti:

- *Cartelle Cliniche Elettroniche (EHR)*: contenenti la storia anamnestica, le visite oncologiche e il tracciamento dei ricoveri;
- *Sistemi Informativi di Anatomia Patologica*: contenenti i referti istologici e la caratterizzazione molecolare del tumore;
- *Registri Farmaceutici e Somministrazioni*: contenenti il dettaglio dei protocolli chemioterapici, immunoterapici e delle linee di trattamento;
- *Laboratori di Analisi e Diagnostica per Immagini*: contenenti la serie storica degli esami ematochimici e le valutazioni strumentali di risposta lesionale.

Tale frammentazione costituisce una barriera sostanziale alla riutilizzabilità secondaria dei dati per scopi di ricerca clinica. Attività fondamentali come l'identificazione di coorti di pazienti con specifiche caratteristiche molecolari, la valutazione dell'efficacia di terapie neoadiuvanti o l'analisi dell'insorgenza di tossicità severe ed eventi avversi richiedono un processo lungo e oneroso di estrazione e riconciliazione manuale delle informazioni.

2.1.2 Standard di Interoperabilità: da HL7 FHIR a RDF

Per superare la rigidità dei silos informativi, la piattaforma CVL adotta un'architettura integrata di trasformazione semantica dei dati [CGR⁺25]. Il punto d'ingresso per la standardizzazione delle informazioni cliniche è rappresentato dallo standard internazionale Fast Healthcare Interoperability Resources (FHIR), sviluppato dall'organizzazione HL7. FHIR definisce un insieme di modelli di dati modulari e tipizzati, denominati *Risorse* (es. *Patient*, *Condition*, *MedicationStatement*, *Observation*), progettati per facilitare lo scambio sicuro di informazioni tra sistemi sanitari eterogenei.

Tuttavia, mentre FHIR eccelle come formato di interscambio operativo via API RESTful, le interrogazioni analitiche su relazioni complesse e multi-livello traggono maggiore beneficio da modelli orientati ai grafi. Per questo motivo, la pipeline di CVL mappa e trasforma le risorse FHIR anonimizzate in un Knowledge Graph interoperabile basato sullo standard W3C Resource Description Framework (RDF) [CGR⁺25]. Questo processo ha consentito la costruzione di una base di conoscenza semantica integrata comprendente i dati clinici, diagnostici e terapeutici di oltre 35.000 pazienti afferenti all'IRST "Dino Amadori".

2.1.3 L'Assistente Virtuale e la Necessità di Dataset Dedicati

Un componente centrale nell'architettura della piattaforma CVL è lo sviluppo di un assistente virtuale basato su Large Language Models (LLMs), concepito per consentire a medici, oncologi e ricercatori di interrogare la base di conoscenza direttamente in linguaggio naturale (italiano o inglese). L'obiettivo primario è eliminare la barriera tecnica rappresentata dal linguaggio SPARQL: per un professionista sanitario, infatti, la formulazione di una query formale richiede non soltanto la padronanza di una sintassi rigida basata su grafi, ma anche la conoscenza approfondita e mnemonica della struttura dell'ontologia, dei nomi specifici dei predicati e dei vocabolari controllati adottati.

Per realizzare tale interfaccia, la semplice adozione di un modello linguistico generico di grande taglia (*off-the-shelf*) o l'affidamento a servizi commerciali via API esterne risultano strategie inapplicabili, per tre ragioni fondamentali:

1. *Mananza di conoscenza ontologica specifica (Ontology Gap)*: Un LLM pre-addestrato su corpora generici non conosce le convenzioni di modellazione interne al Cancer Virtual Lab. Se interpellato su un dominio specialistico, il modello tende ad allucinare predicati plausibili nel linguaggio comune ma inesistenti nel grafo RDF di produzione, generando query sintatticamente corrette ma semanticamente vuote o ineseguibili.
2. *Vincoli di riservatezza e trattamento dei dati sanitari*: Le normative sulla protezione dei dati in ambito medico (quali il GDPR) e la presenza di da-

ti clinici sensibili impongono l'adozione di modelli eseguiti interamente in locale. Ciò preclude l'uso di modelli cloud proprietari e richiede la specializzazione di architetture locali di dimensioni contenute, che necessitano di un addestramento mirato per compensare la minore capacità parametrica generale.

3. *Rigore sintattico ed esecuzione deterministica*: A differenza della generazione di testo naturale, la traduzione *Text-to-SPARQL* non ammette imprecisioni. Anche un singolo errore di formattazione, una parentesi non bilanciata o un identificativo errato rendono l'intera query ineseguibile dall'endpoint del triplestore.

La specializzazione di un modello linguistico locale per questo compito specifico richiede la tecnica del *Supervised Fine-Tuning* (SFT), la quale necessita a sua volta di un dataset di addestramento di alta qualità composto da coppie di domande in linguaggio naturale e query SPARQL corrispondenti.

Tuttavia, un dataset di questo tipo non esiste in partenza per l'ontologia di CVL. La sua costruzione manuale, affidata alla scrittura congiunta di esperti di dominio clinico e di ingegneri del Web Semantico, richiederebbe tempi inaccettabili e non garantirebbe la copertura sistematica delle combinazioni di vincoli possibili. La progettazione e l'implementazione di una pipeline automatizzata in grado di sintetizzare un dataset di addestramento accurato, vario e verificabile costituisce quindi il prerequisito fondamentale e la motivazione primaria del presente lavoro di tesi.

2.2 Knowledge Graph, RDF e Strumenti SPARQL

Per comprendere le scelte architettureali della piattaforma CVL e le sfide legate alla generazione delle query, è necessario formalizzare il modello di dati sottostante. L'organizzazione delle informazioni cliniche non si basa su tabelle relazionali tradizionali, ma sul paradigma dei Knowledge Graphs (KGs) e sugli standard del Web Semantico raccomandati dal W3C.

2.2.1 Dai Dati Strutturati ai Knowledge Graph

Un KG è una struttura dati orientata a grafo, progettata per rappresentare la conoscenza integrando entità, attributi e relazioni. A differenza delle basi di dati relazionali tradizionali, dove i dati sono vincolati a schemi rigidi suddivisi in tabelle e chiavi esterne, un Knowledge Graph esprime la conoscenza in modo flessibile ed esplicito:

- *I Nodi* rappresentano le entità concrete del dominio (es. un specifico paziente, una diagnosi, un farmaco);
- *Gli Archi Diretti* rappresentano le relazioni semantiche tipizzate che collegano un nodo all'altro (es. `ha_diagnosi`, `assume_farmaco`).

Questo approccio risulta particolarmente vantaggioso in ambito oncologico, dove la struttura dei dati è intrinsecamente complessa e soggetta a continua evoluzione, dove nuove proprietà o relazioni possono essere introdotte nel grafo senza dover ridefinire lo schema globale dell'intera base di dati.

2.2.2 Resource Description Framework (RDF)

Lo standard formale impiegato per concretizzare un Knowledge Graph è il Resource Description Framework (RDF). In RDF, qualsiasi informazione o asserzione viene scomposta nella sua unità elementare atomica chiamata *tripla*:

(Soggetto, Predicato, Oggetto)

Ciascun elemento della tripla svolge un ruolo preciso nella rete di conoscenza:

- *Soggetto*: identifica l'entità principale dell'asserzione mediante un Internationalized Resource Identifier (IRI), un indirizzo univoco a livello globale;
- *Predicato*: esprime la relazione specifica o la proprietà descritta, anch'esso identificato da un IRI ;
- *Oggetto*: rappresenta il punto di arrivo della relazione. Può essere a sua volta un IRI che punta a un'altra entità del grafo (dando origine a una struttura

di nodi interconnessi) oppure un valore letterale (*Literal*) tipizzato (es. una stringa, un numero intero, una data ISO 8601 o un valore booleano).

L'insieme di tutte le triple forma il grafo RDF. Le regole formali, i vocabolari controllati e la gerarchia di classi ammesse all'interno del grafo costituiscono l'*ontologia* del sistema, ovvero lo schema concettuale del dominio clinico.

2.2.3 SPARQL

Per estrarre informazioni e interrogare un grafo RDF si utilizza SPARQL (*SPARQL Protocol and RDF Query Language*), il linguaggio di interrogazione standard per i dati del Web Semantico.

A differenza di SQL, che opera eseguendo operazioni di join tra tabelle, SPARQL funziona tramite il meccanismo del *graph pattern matching* (corrispondenza di schemi di grafo). La clausola **WHERE** di una query SPARQL definisce un insieme di "triple con incognite" (dette *triple pattern*), in cui alcuni elementi sono sostituiti da variabili identificate dal prefisso ? (es. ?**patient**, ?**diagnosis**). L'algoritmo di esecuzione confronta lo schema richiesto con il grafo RDF reale, individuando tutti i sotto-grafi che soddisfano simultaneamente le combinazioni di vincoli specificate.

Per raffinare la ricerca, SPARQL mette a disposizione clausole espressive quali:

- **FILTER**: applica vincoli logici, algebrici o temporali sui valori letterali (es. restringere la ricerca a pazienti nati entro un determinato intervallo di date);
- **VALUES**: limita i valori assegnabili a una variabile a un elenco discreto e prefissato;
- **OPTIONAL**: consente di estrarre attributi accessori senza compromettere la restituzione dei risultati principali qualora il dato non sia presente nel grafo per un determinato paziente.

2.2.4 Validazione Sintattica Determinista con RDFLib

Poiché le query SPARQL generate dalla pipeline sintetica dovranno servire da target per l'addestramento di un modello di linguaggio, la presenza di errori di sintassi comprometterebbe l'efficacia dell'apprendimento.

Per garantire la validità formale delle query senza dover sovraccaricare l'endpoint SPARQL di produzione con continue chiamate di rete, il sistema sfrutta la libreria Python `rdflib`. `rdflib` fornisce un parser sintattico deterministico che analizza l'albero sintattico astratto (*AST*) della query generata in locale. Qualora la query presenti parentesi non bilanciate, keyword errate o sintassi SPARQL 1.1 non conforme, il parser solleva un'eccezione, consentendo alla pipeline di scartare o rigenerare immediatamente l'esempio difettoso.

2.3 Architettura Software e Pattern

La progettazione di pipeline complesse per l'elaborazione di dati e la sintesi di corpora richiede l'adozione di modelli di programmazione e strumenti software in grado di garantire modularità, manutenibilità e rapidità di ispezione. In questa sezione vengono analizzati, a livello teorico e metodologico, i principali pattern di progettazione ad oggetti e i paradigmi di prototipazione rapida considerati nell'ambito del progetto.

2.3.1 Il Pattern Architetturale Mixin

Nello sviluppo di sistemi orientati agli oggetti, la gestione di processi complessi multi-stadio richiede una rigorosa separazione delle responsabilità per rispettare i principi SOLID, con particolare riferimento al principio di singola responsabilità (*Single Responsibility Principle*). Quando una singola classe accumula funzionalità eterogenee, quali la gestione della comunicazione di rete, il parsing dei dati, la logica di campionamento e le operazioni di I/O su disco, la struttura del codice tende a diventare monolitica, rigida e complessa da sottoporre a test.

Per risolvere tale criticità senza ricorrere ad architetture sproporzionatamente articolate, la programmazione orientata agli oggetti mette a disposizione il pattern *Mixin*. Un Mixin si definisce come una classe progettata non per rappresentare una struttura di dati autonoma, bensì per fornire un insieme circoscritto e coerente di metodi a un'altra classe mediante l'impiego dell'ereditarietà multipla. A differenza dell'ereditarietà classica, la quale esprime una relazione di specializzazione fra tipi, il Mixin introduce una relazione di estensione funzionale.

Tale approccio comporta tre proprietà fondamentali sul piano teorico. In primo luogo, la classe *Mixin* non viene mai istanziata direttamente nel flusso di esecuzione. In secondo luogo, essa non mantiene uno stato interno complesso o un costruttore autonomo, bensì opera sui metodi e sugli attributi messi a disposizione dalla classe che la eredita. In terzo luogo, la classe principale, detta orchestratrice, può ereditare da più *Mixin* contemporaneamente, incorporandone le capacità all'interno di un'unica interfaccia unificata.

I vantaggi metodologici derivanti da questa architettura risiedono nella scomposizione del dominio applicativo in unità logiche isolate, nella garanzia di un'elevata coesione interna unita a un basso accoppiamento fra le parti e nella possibilità di riutilizzare i medesimi componenti in configurazioni differenti senza duplicazione di codice.

2.3.2 Prototipazione e Interfacce Reattive con Streamlit

Nelle fasi di calibrazione dei parametri di un algoritmo e di analisi esplorativa dei dati, l'interazione programmata da riga di comando presenta limiti di usabilità, poiché non offre un riscontro visivo immediato sullo stato dell'elaborazione o sulla struttura delle query prodotte.

Per ovviare a questo limite, la letteratura e la pratica ingegneristica orientata alla *Data Science* incoraggiano l'adozione di framework per la prototipazione rapida di interfacce web reattive, tra i quali figura **Streamlit**. A differenza delle architetture web tradizionali, che richiedono la separazione tra un servizio di backend e un'interfaccia frontend sviluppata con tecnologie dedicate, Streamlit si basa su un paradigma di esecuzione reattiva direttamente in linguaggio Python. Ogni interazione dell'utente con gli elementi di controllo dell'interfaccia determina un aggiornamento dello stato di sessione, consentendo di parametrizzare gli algoritmi, gestire eventi di interruzione e visualizzare i dati in tempo reale.

Poiché la piattaforma del Cancer Virtual Lab risulta interamente sviluppata mediante l'impiego di Streamlit, il primo stadio della pipeline di generazione è stato concepito per integrarsi nativamente all'interno dell'infrastruttura preesistente. Questa scelta architetturale ha permesso di estendere il sistema con un modu-

lo di controllo avanzato, consentendo il monitoraggio interattivo dei processi di generazione e la sincronizzazione con l'ambiente operativo del progetto.

2.4 Lavori Correlati e Benchmark di Text-to-SPARQL

La costruzione di dataset di addestramento e benchmark per il compito di *Text-to-SPARQL* ha seguito un'evoluzione significativa negli ultimi anni, passando da corpora puramente sintetici a metodologie guidate da modelli linguistici. L'analisi dello stato dell'arte consente di classificare le tecniche di generazione in tre approcci principali, evidenziandone i limiti strutturali rispetto alle esigenze di un dominio specialistico come quello clinico-oncologico.

2.4.1 Approcci Basati su Template Rigidi e Corpora Sintetici

I primi approcci alla generazione di dataset per il Web Semantico si sono basati sull'impiego di template fissi associati a regole grammaticali formali. Benchmark pubblici di riferimento, quali LC-QuAD [TMDL17] e la sua successiva evoluzione LC-QuAD 2.0 [DBAL19], hanno adottato metodologie per la generazione su larga scala di coppie domanda-query mirate a basi di conoscenza generali come DBpedia e Wikidata.

In questo approccio, un insieme di modelli linguistici pre-strutturati viene riempito iterativamente con istanze estratte dal grafo. Sebbene tali dataset offrano volumi elevati di dati, presentano limiti strutturali insormontabili quando applicati a ontologie verticali: la rigidità dei template genera formulazioni linguistiche artificiali e ripetitive, incapaci di coprire la varietà lessicale e la complessità relazionale tipica del linguaggio umano specialistico-oncologico.

2.4.2 Sintesi Guidata da Regole e Grammatiche Formali (MK-SQuIT)

Per superare la rigidità dei template tradizionali, successivi lavori hanno esplorato tecniche di sintesi automatizzata basate sulla generazione orientata alla gram-

tica, come ad esempio la metodologia MK-SQuIT [SCKS20]. Questo approccio sfrutta grammatiche formali ed elementi di logica per assemblare in modo ricorsivo piccoli frammenti di frase con i rispettivi vincoli della query, costruendo così domande strutturalmente più articolate.

Sebbene tali sistemi migliorino la copertura delle combinazioni logiche, mantengono un forte legame con la struttura formale sottostante. Le domande prodotte tendono a rispecchiare fedelmente la struttura della query SPARQL anziché il modo in cui un operatore umano formulerebbe il quesito, risultando inadeguate ad addestrare modelli linguistici destinati a un'utenza clinica non tecnica.

2.4.3 Generazione Indotta da Large Language Model

Con l'avvento dei Large Language Model, la letteratura recente ha esplorato l'uso diretto di modelli generativi per la sintesi e l'arricchimento di dataset Text-to-SPARQL, come dimostrato da studi applicati in contesti aziendali e generali [LdCF25]. In questo paradigma, un LLM viene interpellato per parafrasare o generare direttamente domande a partire da query formali.

Ciononostante, l'affidamento incondizionato a un LLM per la sintesi end-to-end introduce criticità rilevanti:

- *Allucinazioni semantiche*: il modello tende a inventare relazioni o classi inesistenti nell'ontologia di riferimento;
- *Alterazione dei valori letterali*: identificativi, date e codici clinici critici vengono frequentemente modificati o omessi dal modello durante la generazione del testo;
- *Assenza di controlli strutturali*: l'assenza di un collegamento deterministico tra la query SPARQL e il prompt generato rende impossibile garantire la correttezza semantica senza un'accurata validazione esterna.

2.4.4 Motivazioni della Scelta Metodologica nel Contesto CVL

L'analisi dei lavori correlati evidenzia come nessuno degli approcci tradizionali sia direttamente applicabile al Cancer Virtual Lab. I benchmark generali non coprono l'ontologia oncologica IRST, i template rigidi falliscono sulla varietà linguistica e la generazione non vincolata tramite LLM espone al rischio inaccettabile di corruzione dei dati clinici.

Per tali ragioni, questo lavoro adotta una strategia ibrida e controllata. Un primo stadio impiega un generatore combinatorio basato sullo schema del grafo per garantire la correttezza formale delle query; a questo fa seguito un secondo stadio di augmentation linguistica guidato da LLM, rigorosamente vincolato da un meccanismo di mascheramento dei valori letterali.

2.5 Fine-tuning di Large Language Model

I Large Language Model (LLM) pre-addestrati su corpora generici possiedono una vasta conoscenza linguistica di base, ma spesso mostrano prestazioni insufficienti quando applicati a compiti specialistici che richiedono il rispetto di sintassi rigide o di ontologie di dominio, come la traduzione *Text-to-SPARQL* trattata in questa tesi. Per adattare un modello pre-addestrato a un compito specifico si ricorre alla tecnica del *fine-tuning*.

2.5.1 Adattamento supervisionato (SFT)

Il *Supervised Fine-Tuning* (SFT) consiste nel prosieguito dell'addestramento del modello su un dataset specifico composto da coppie di input e output desiderati. Durante questo processo, i pesi della rete neurale vengono aggiornati tramite retropropagazione dell'errore per minimizzare la discrepanza tra la sequenza generata e quella target, insegnando al modello la struttura sintattica del linguaggio di destinazione e la semantica dei predicati dell'ontologia. La scelta di *quanti* e *quali* parametri aggiornare durante questo processo distingue diverse strategie di fine-tuning, descritte di seguito.

2.5.2 Full Fine-Tuning

Nella sua forma più diretta, il fine-tuning aggiorna la totalità dei parametri del modello pre-addestrato, esattamente come avviene durante il pre-addestramento originale, ma su un dataset più piccolo e specifico. Questo approccio, detto *Full Fine-Tuning*, offre in linea di principio la massima capacità di adattamento, poiché nessun parametro resta vincolato al proprio valore originale.

Il costo computazionale è però proporzionalmente elevato. Oltre allo spazio necessario per i pesi del modello stesso, l'ottimizzatore deve mantenere in memoria stati aggiuntivi per ciascun parametro addestrabile (ad esempio, gli ottimizzatori della famiglia Adam richiedono due valori di momento per ogni peso), moltiplicando di fatto i requisiti di memoria rispetto alla sola dimensione del modello. Per modelli di grandi dimensioni, questo può rendere il Full Fine-Tuning impraticabile su hardware di classe consumer o con risorse di calcolo limitate.

2.5.3 Tecniche di adattamento efficiente (PEFT, LoRA e QLoRA)

Per ovviare a questo limite, sono state introdotte le tecniche di *Parameter-Efficient Fine-Tuning* (PEFT), che aggiornano solo una piccola porzione dei parametri del modello, o ne introducono di nuovi in quantità ridotta, lasciando la maggior parte dei pesi originali invariata.

Tra le metodologie PEFT più diffuse figura *Low-Rank Adaptation* (LoRA) [HSW⁺22] che, invece di modificare la matrice dei pesi originali $W_0 \in \mathbb{R}^{d \times k}$ del modello, approssima l'aggiornamento dei pesi ΔW tramite il prodotto di due matrici a rango ridotto $A \in \mathbb{R}^{r \times k}$ e $B \in \mathbb{R}^{d \times r}$, dove il rango $r \ll \min(d, k)$:

$$W = W_0 + \Delta W = W_0 + B \cdot A$$

Solo le matrici A e B , molto più piccole della matrice originale, vengono addestrate e mantenute in memoria con i relativi stati dell'ottimizzatore; W_0 resta congelata e viene caricata una sola volta. Questo approccio riduce drasticamente il numero di parametri addestrabili (spesso di oltre il 99%), abbattendo i requisiti di memoria rispetto al Full Fine-Tuning, a fronte di una capacità di adattamento inferiore in

linea di principio, poiché l'aggiornamento è vincolato a un sottospazio di rango basso.

L'evoluzione *Quantized LoRA* (QLoRA) estende questo principio quantizzando il modello di base congelato a una precisione di 4-bit (utilizzando il formato *NormalFloat4*), riducendo ulteriormente la memoria occupata da W_0 stessa e non solo dagli stati dell'ottimizzatore, consentendo di eseguire l'addestramento di LLM di grandi dimensioni su hardware di classe consumer o su singola GPU locale.

2.5.4 Funzione di perdita e formato delle istruzioni

Durante il fine-tuning per compiti di generazione di codice o query formali, i dati vengono tipicamente strutturati in formato istruzione (*Instruction Format*), dove il prompt di input definisce il contesto o il ruolo del sistema e l'output target rappresenta la sequenza da generare.

La funzione di perdita usata è la *Cross-Entropy Loss* calcolata sui token sequenziali. Per ottimizzare l'addestramento ed evitare che il modello spenda capacità computazionale per imparare a ripetere il prompt di input, si applica comunemente il mascheramento della loss sui token di input (*Data Collator for Completion Only*): la perdita viene calcolata ed elaborata esclusivamente sui token corrispondenti all'output target, forzando il modello a concentrare l'apprendimento unicamente sulla generazione della struttura formale corretta.

Capitolo 3

Analisi

3.1 Obiettivi

L'obiettivo primario del progetto è la creazione di una pipeline in grado di costruire un dataset annotato per il fine-tuning di un modello linguistico di dimensioni contenute, destinato alla traduzione di domande in linguaggio naturale in query SPARQL da eseguire sul knowledge graph del Cancer Virtual Lab. Da questo obiettivo generale derivano quattro obiettivi più specifici, che hanno guidato le scelte progettuali descritte nei capitoli successivi.

- O1** *Copertura selettiva ed estendibilità.* Il dataset deve coprire le parti dello schema del knowledge graph più rilevanti per l'applicazione, senza richiedere la copertura completa dello schema. Il sistema deve inoltre permettere di estendere progressivamente questa copertura aggiungendo nuove classi e relazioni, senza richiedere una riprogettazione del generatore.
- O2** *Varietà linguistica autentica.* Le domande generate devono presentare una varietà linguistica sufficiente a evitare formulazioni ripetitive o riconducibili a un unico template, mantenendo invariato il significato espresso dalla query sottostante.
- O3** *Tracciabilità completa.* Ogni domanda generata deve essere associata in modo univoco alla query SPARQL da cui deriva. L'associazione tra doman-

da e query deve essere quindi conservata esplicitamente, così da rendere verificabile ogni coppia utilizzata nel dataset.

- O4** *Volume di dati adeguato al fine-tuning.* Il dataset deve contenere un numero di esempi sufficiente per supportare il fine-tuning del modello linguistico considerato e deve garantire una copertura adeguata delle diverse tipologie di query previste.

3.2 Vincoli

Il progetto è stato condotto sotto un insieme di vincoli tecnici che ne hanno influenzato le scelte architettoniche, descritte nel dettaglio nei capitoli successivi.

- V1** *Compatibilità con uno schema esistente e non modificabile.* Il sistema deve generare query conformi allo schema dell'ontologia del Cancer Virtual Lab, preesistente al progetto e non alterabile nel suo ambito. Nomi di classi, proprietà e relazioni sono vincoli esterni, non negoziabili in funzione di una generazione più semplice o di una copertura più ampia.
- V2** *Trattamento di dati clinici sensibili.* I valori coinvolti nelle query come identificativi di pazienti, codici di classificazione clinica o date sono dati clinici il cui significato deve rimanere verificabile e inalterato in ogni fase del processo. Questo vincolo esclude, inoltre, l'invio di tali dati a servizi esterni non controllati, imponendo l'esecuzione locale di qualunque componente del sistema che vi acceda.
- V3** *Capacità limitata del modello linguistico impiegato.* A causa del vincolo sull'esecuzione locale imposto da V2, il sistema potrà sfruttare solo modelli di dimensioni molto contenute, non potendo quindi fare affidamento sulla correttezza semantica spontanea dell'output prodotto, e richiede un meccanismo di validazione esplicito e indipendente dal modello stesso.

3.3 Requisiti funzionali

I requisiti funzionali descrivono cosa il sistema deve produrre, indipendentemente da come tale produzione venga realizzata.

- RF1** *Generazione di coppie prompt-query.* Il sistema deve generare coppie costituite da una query SPARQL sintatticamente valida e da un prompt in linguaggio naturale corrispondente, a partire dallo schema del knowledge graph.
- RF2** *Produzione di varianti multiple.* Il sistema deve produrre, per ciascuna coppia generata, un numero variabile di varianti parafrasate del medesimo prompt, preservando invariata la query associata.
- RF3** *Preservazione dei valori letterali.* Il sistema deve garantire, in ciascuna variante parafrasata, la presenza dei valori letterali presenti nella query, quali identificativi di pazienti, codici di classificazione clinica o date, nella loro forma esatta o in una forma alternativa verificabile.
- RF4** *Complessità variabile delle query.* Il sistema deve supportare la generazione di query di complessità variabile, dal caso di un singolo vincolo su una singola proprietà fino a query che attraversano più relazioni e combinano più vincoli sullo stesso nodo.

3.4 Requisiti non funzionali

I requisiti non funzionali riguardano invece proprietà trasversali del sistema, non direttamente osservabili nel singolo output prodotto.

- RNF1** *Sostenibilità computazionale.* Il sistema deve garantire un tasso di accettazione delle generazioni sufficientemente elevato da rendere il processo economicamente sostenibile in termini di tempo di calcolo.
- RNF2** *Resilienza a interruzioni.* A causa del vincolo sull'esecuzione locale imposto da V2, che comporta risorse di calcolo limitate, tempi di elaborazione prolungati e un maggior rischio di interruzioni, il sistema deve adottare meccanismi

che permettano di limitare la perdita di lavoro già svolto in caso di arresto imprevisto.

RNF3 *Robustezza semantica.* Il sistema deve limitare il rischio che la variazione linguistica modifichi, senza rilevarlo, il significato della domanda rispetto alla query sottostante, con particolare attenzione ai valori clinici controllati.

Nota *La robustezza semantica riguarda quindi la fedeltà del contenuto, mentre la tracciabilità di O3 riguarda la provenienza e l'associazione tra testo e query.*

Capitolo 4

Progettazione

4.1 Analisi dei problemi

I requisiti e i vincoli descritti nel capitolo precedente si traducono in tre sotto-problemi progettuali distinti, la cui interazione determina la difficoltà complessiva del compito. La distinzione è utile perché consente di separare le difficoltà legate alla costruzione delle query da quelle introdotte dalla variazione linguistica e, infine, dalla necessità di verificare gli output prodotti.

P1 *Sotto-problema combinatorio.* Dato lo schema del knowledge graph, generare query SPARQL sintatticamente corrette che coprano una distribuzione rappresentativa dell'uso reale del sistema, senza produrre un numero di combinazioni non gestibile nei tempi o eccessivo per un corretto fine-tuning.

P2 *Sotto-problema linguistico.* Dato un prompt in forma template privo di variazione, generare un numero variabile di parafrasi che siano al tempo stesso fluenti in linguaggio naturale e fedeli al contenuto semantico della query sottostante. A differenza di P1, qui la difficoltà non risiede nella dimensione dello spazio delle soluzioni possibili, ma nell'assenza di un criterio puramente formale per distinguere una parafrasi corretta da una scorretta. Una parafrasi può infatti essere grammaticalmente ineccepibile e superare ogni controllo meccanico applicabile, e tuttavia alterare il significato della domanda originale.

P3 *Sotto-problema di validazione.* Distinto da P1 e P2 ma dipendente da entrambi: verificare, per ciascuna variante generata, che la relazione con la query associata sia corretta e che il contenuto semantico non sia stato alterato durante la variazione linguistica. La tracciabilità richiesta da O3 è una proprietà verificabile della provenienza del dato; la robustezza semantica di RNF3 introduce invece una difficoltà ulteriore, poiché non esiste un criterio puramente formale che stabilisca con certezza l'equivalenza tra una formulazione in linguaggio naturale e la rappresentazione SPARQL sottostante. P3 è quindi il punto in cui la pipeline deve combinare controlli strutturali e semantici, ed è discusso nel dettaglio nel capitolo di valutazione.

I tre sotto-problemi individuati nell'analisi vengono affrontati da componenti distinti della pipeline. P1 è affrontato attraverso il generatore parametrico di query, che consente di controllare lo spazio delle configurazioni esplorate. P2 è affrontato attraverso la fase di augmentation linguistica, progettata per produrre formulazioni diverse a partire dalla medesima query. P3 richiede invece una fase di validazione, che verifica gli output prodotti dalle fasi precedenti prima del loro inserimento nel dataset.

4.2 Design del generatore

Questa sezione descrive le decisioni progettuali alla base del primo stadio del sistema, responsabile della generazione strutturata delle coppie prompt-query.

4.2.1 La pipeline di generazione

Generare una singola coppia prompt-query valida a partire da un'entità dello schema richiede diversi passaggi logicamente distinti tra loro. Occorre prima sapere quali proprietà e relazioni siano disponibili per quell'entità, poi individuare i percorsi che permettono di raggiungerle nel grafo, poi scegliere quali di quelle proprietà vincolare rispettando i limiti imposti, poi tradurre questa scelta in una struttura di query, poi decidere con quali valori concreti riempirla, poi trasformare il tutto in un testo in linguaggio naturale, e infine verificare che il risultato ottenuto sia

davvero corretto. Concatenare tutti questi passaggi in un'unica procedura indifferenziata renderebbe il codice difficile da comprendere, da testare e da modificare, perché un cambiamento riguardante anche un solo aspetto, per esempio il modo in cui si sceglie un valore, rischierebbe di intrecciarsi con la logica di tutti gli altri.

Il generatore struttura quindi questo processo come una pipeline di fasi ben definite, ciascuna responsabile di una trasformazione specifica, che riceve in ingresso il risultato della fase precedente e produce l'input per quella successiva.

Questa scomposizione rende ciascuna fase comprensibile e verificabile in isolamento, senza dover ragionare sull'intero processo per capire o modificare una singola trasformazione. Apre inoltre la strada, in fase di implementazione, a una corrispondenza diretta tra ciascuna fase concettuale qui definita e il componente software responsabile di realizzarla (sezione 5.1.1).

4.2.2 Modello del dominio

Il modello di dominio di questo progetto non coincide con l'intero schema del knowledge graph del Cancer Virtual Lab, ma con un sottoinsieme di 45 classi fornito dall'IRST. La selezione non ha seguito un criterio puramente tecnico, quale l'inclusione di ogni classe disponibile nel triplestore, ma è stata guidata dagli obiettivi clinici definiti a monte del progetto. Questo sottoinsieme, e la struttura relazionale che lo lega, costituisce un dato di fatto imposto dal knowledge graph ricevuto, non una scelta progettuale: le decisioni discusse nel resto di questa sezione riguardano non quali classi includere, ma come organizzarle e pesarle ai fini della generazione.

Le quarantacinque classi sono organizzate in gruppi di priorità decrescente, come in figura 4.1, secondo un criterio di distanza strutturale dal nodo Patient. L'adozione di questo criterio risponde a un'osservazione sul dominio applicativo stesso: Patient costituisce la radice naturale del grafo clinico, coerentemente con il modo in cui un clinico formula una domanda, ovvero a partire dal paziente, non da una classe periferica quale può essere un singolo biomarcatore. La distanza dal nodo radice offre, in aggiunta, un criterio oggettivo e calcolabile direttamente dalla struttura del grafo, senza richiedere l'imposizione anticipata di un giudizio soggettivo su quale classe sia più rilevante dal punto di vista clinico. Tale distanza si è inoltre rivelata un proxy efficace per la complessità della query generabile:

una classe raggiungibile con un solo passaggio relazionale produce tipicamente domande semplici e dirette, mentre una classe raggiungibile con due o tre passaggi richiede query che concatenano più relazioni, rendendo la distanza informativa non solo rispetto alla rilevanza clinica, ma anche rispetto alla difficoltà attesa della domanda-tipo associata a ciascuna classe.

L'assenza di un simile criterio di priorizzazione avrebbe comportato conseguenze dirette sul dataset prodotto, discusse in relazione a O4 nel capitolo di analisi. Trattando le quarantacinque classi come equivalenti, il sistema avrebbe allocato un budget di generazione identico a ciascuna, indipendentemente dalla sua centralità nel dominio clinico. Classi concettualmente centrali, quali Patient, Diagnosis e Therapy, avrebbero ricevuto una rappresentazione insufficiente rispetto alla frequenza con cui compaiono nell'uso reale, mentre classi periferiche e altamente granulari, quali le singole categorie della classificazione TNM, avrebbero prodotto un numero di esempi ridondante rispetto alla loro effettiva rilevanza. Il risultato atteso di un simile sbilanciamento non sarebbe stato un dataset semplicemente meno efficiente, ma un modello allenato su di esso incline a generalizzare peggio proprio sulle categorie di domande più frequenti nell'uso previsto del sistema.

La distanza dal nodo Patient non costituisce l'unico criterio di raggruppamento adottato nel corso del progetto. Un secondo raggruppamento, tematico anziché strutturale, organizza le medesime quarantacinque classi per area clinica (demo-grafia, stadiazione, trattamento, ecc.) ed è stato introdotto in una fase successiva del progetto, discussa nel capitolo di implementazione. I due criteri non sono stati concepiti come alternativi, ma come complementari; di fatti la distanza strutturale fornisce una progressione di complessità pulita, adatta a un primo passaggio di generazione, mentre il raggruppamento tematico ricombina le stesse classi secondo relazioni di co-occorrenza diverse, consentendo a un secondo passaggio di generazione di produrre query multi-relazionali strutturalmente nuove, anziché ricampionare gli stessi pattern già coperti dal primo raggruppamento.

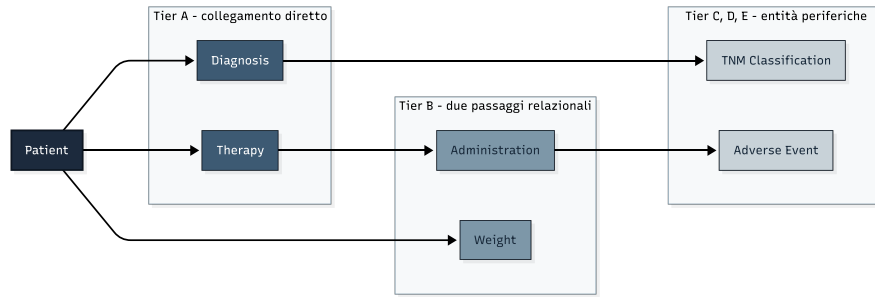


Figura 4.1: Modello di dominio semplificato: Patient come nodo centrale, con una classe rappresentativa per ciascuno dei sei gruppi di priorità

Il numero di record generati per ciascun gruppo decresce dal centro alla periferia del dominio ma tale decrescita non segue una progressione lineare rigida. Questo andamento è coerente con una calibrazione empirica, ovvero condotta bilanciando l'equilibrio finale del dataset, piuttosto che con l'applicazione di una formula matematica definita anticipatamente. Lo stesso principio di calibrazione empirica viene applicato al sistema dei quattro parametri descritto nella sezione seguente.

In sintesi, il dataset text-to-SPARQL prodotto da questo sistema non è generato in modo uniforme sull'intero schema disponibile, ma segue una gerarchia di rilevanza clinica propria del progetto. Le classi incluse e i parametri di complessità applicati a ciascuna sono stati scelti con l'obiettivo di riflettere le tipologie di domande ritenute più rilevanti nel contesto del Cancer Virtual Lab, privilegiando la rilevanza del dominio rispetto alla massimizzazione della copertura formale del grafo.

4.2.3 Sistema dei parametri

Il modello di dominio descritto nella sezione precedente definisce quali classi partecipano alla generazione e con quale peso relativo, ma non specifica in alcun modo la complessità delle singole query generabili su ciascuna di esse.

Il generatore espone quattro parametri indipendenti, ciascuno regolabile entro un intervallo minimo e massimo, la cui combinazione determina la complessità della query risultante:

- il numero di relazioni attraversate nel grafo a partire dal nodo radice;
- il numero di percorsi alternativi combinati nella medesima query tramite clausole di disgiunzione;
- il numero di proprietà vincolate su un medesimo nodo;
- il numero di valori ammessi per ciascun vincolo.

Questi quattro parametri corrispondono a dimensioni strutturalmente distinte lungo cui una query SPARQL può crescere in complessità. La loro separazione consente di variare un aspetto della complessità senza doverlo necessariamente far crescere insieme agli altri. Trattare la complessità come una singola grandezza scalare avrebbe nascosto questa distinzione, impedendo un controllo mirato su quale aspetto specifico della query si desidera far variare per una determinata classe o gruppo di classi.

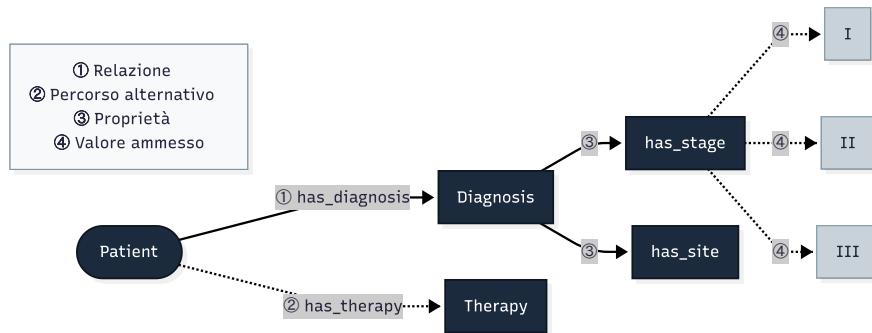


Figura 4.2: Modello esemplificativo dei parametri filtrabili

Sebbene l'impianto concettuale dei quattro parametri sia rimasto invariato nel corso del progetto, i valori numerici effettivamente assegnati a ciascuno di essi non sono stati definiti una sola volta, ma calibrati progressivamente attraverso diverse configurazioni successive, tutte vincolate da un limite superiore imposto dall'endpoint SPARQL stesso. Infatti query eccessivamente complesse, al crescere del numero di relazioni e proprietà vincolate simultaneamente, incorrevano in timeout o fallivano l'esecuzione contro il triplestore reale, rendendo necessario mantenere ciascun parametro entro un tetto massimo compatibile con le capacità dell'endpoint.

4.2.4 Meccanismo di estensione

Il sistema di parametri descritto nella sezione precedente offre, come conseguenza diretta della sua struttura, una capacità che va oltre la sola configurazione iniziale della complessità: la possibilità di ampliare il dataset generato senza rigenerare le porzioni già prodotte, e senza il rischio che le nuove query si sovrappongano a quelle esistenti.

Questa capacità è resa possibile dal fatto che un intervallo minimo-massimo definisce uno spazio di configurazioni delimitato con precisione. Impostando, per un nuovo preset di generazione, un intervallo disgiunto da quello di un preset precedente su almeno una dimensione rilevante, i due spazi risultano disgiunti a livello di configurazione dei parametri. In questo modo il nuovo preset esplora combinazioni che non appartenevano allo spazio configurato in precedenza. La proprietà è quindi garantita a livello parametrico; l'assenza di duplicati a livello di query è invece una proprietà che dipende dal modo in cui le configurazioni vengono trasformate in query e, se necessario, può essere verificata esplicitamente durante la generazione.

Questa distinzione rende il meccanismo di estensione più preciso: l'aggiunta di nuove porzioni al dataset non richiede una riprogettazione del generatore, ma la definizione di nuovi preset i cui intervalli siano scelti in relazione a quelli già utilizzati. La procedura può essere applicata ripetutamente, mantenendo traccia dello spazio parametrico già esplorato e introducendo nuove configurazioni al di fuori di esso.

4.3 Design dell'augmentation

Questa sezione descrive le decisioni progettuali alla base del secondo stadio del sistema, responsabile della trasformazione delle coppie prompt-query prodotte dal generatore in più varianti linguistiche naturali, secondo il compromesso tra varietà e fedeltà semantica richiesto da O2 e la robustezza semantica richiesta da RNF3.

4.3.1 Mascheramento dei valori letterali

Il problema affrontato da questo stadio è duplice: da un lato produrre varie riformulazioni linguisticamente a partire da uno stesso prompt, dall'altro garantire che tale variazione non alteri i valori letterali che compaiono nel prompt e che sono referenziati dalla query SPARQL associata (identificativi, codici di classificazione clinica, date). Lasciare che un modello linguistico riscriva liberamente l'intero testo, valori inclusi, esporrebbe il sistema al rischio già discusso in RF3 e RNF3, ovvero una riformulazione apparentemente valida ma silenziosamente scorretta rispetto alla query originale.

La soluzione adottata separa la variazione linguistica dai valori da preservare tramite un meccanismo di mascheramento. Prima di essere sottoposto al modello, ogni valore letterale identificato nel prompt viene sostituito da un token segnaposto opaco; il modello riformula quindi il testo mascherato, e solo al termine della generazione i segnaposto vengono sostituiti nuovamente con i valori originali. In questo modo il modello non ha mai accesso al valore letterale reale durante la riformulazione, e può quindi soltanto ricollocarlo all'interno di una struttura sintattica diversa.

Sono stati introdotti inoltre due accorgimenti specifici per la gestione dei valori mascherati. Il primo riguarda quando un segnaposto rappresenta una condizione vera o falsa, come la presenza di un evento avverso; in questo caso il valore non viene mantenuto nella forma letterale *true* o *false*, ma viene convertito in una formulazione più naturale per il linguaggio umano, come «con» o «senza», seguita dal termine clinico corrispondente.

Il secondo accorgimento riguarda i record caratterizzati da un numero elevato di valori mascherati. In questi casi, il testo non viene sottoposto al modello attraverso un'unica chiamata, ma viene suddiviso in più porzioni, ciascuna contenente al massimo un numero prefissato di segnaposto. Le singole porzioni vengono quindi riformulate separatamente e ricomposte al termine della generazione. Questa scelta deriva da un'osservazione empirica secondo cui, oltre una determinata soglia di segnaposto presenti nello stesso prompt, la qualità delle riformulazioni generate tende a diminuire.

4.3.2 Generazione iterativa e validazione

Il mascheramento garantisce che i valori letterali, se presenti nell'output, siano corretti; non garantisce però che il modello produca sempre un output valido al primo tentativo, né che varianti successive dello stesso record non finiscano per ripetersi tra loro, vanificando l'obiettivo di varietà linguistica di O2.

Il sistema affronta questo problema attraverso un ciclo iterativo di generazione e verifica. Ogni candidato prodotto dal modello viene sottoposto a una fase di validazione che considera diversi criteri.

- **Preservazione dei segnaposto.** Il candidato viene rifiutato se anche un solo segnaposto è stato omesso, modificato o sostituito.
- **Assenza di segnaposto non previsti.** Viene verificato che il testo non contenga sequenze che imitano la sintassi di un segnaposto senza corrispondere a uno dei segnaposto effettivamente presenti nel testo originale.
- **Controllo della similarità.** Il candidato viene confrontato con le varianti già accettate per lo stesso record. Se risulta eccessivamente simile a una di esse, viene rifiutato. La soglia di similarità viene resa progressivamente più permissiva al diminuire della lunghezza del testo, così da evitare che formulazioni brevi vengano considerate erroneamente come duplicati.

In caso di rifiuto, il sistema non si limita a ripetere la generazione. Il candidato scartato e il relativo motivo vengono forniti al modello come contesto aggiuntivo per il tentativo successivo, in modo da ridurre la probabilità che venga ripetuto lo stesso errore. A ogni nuovo tentativo viene inoltre incrementata progressivamente la temperatura di campionamento, favorendo la produzione di una formulazione diversa da quella precedente.

Se anche l'ultimo tentativo disponibile viene rifiutato, il sistema non forza comunque l'inserimento di una nuova variante. Viene invece mantenuto il testo originale non modificato e viene registrato esplicitamente il ricorso a questo comportamento di ripiego. In questo modo si evita di accettare silenziosamente una formulazione che violi i vincoli definiti da RF3 o RNF3.

Il numero di varianti richieste per ciascun record non è inoltre fissato in modo uniforme, ma proporzionato al numero di valori mascherati nel record: un prompt

con pochi valori offre in pratica poco margine di riformulazione prima di esaurire le combinazioni naturali, per cui imporre lo stesso numero di varianti richieste a un record semplice e a uno complesso avrebbe prodotto, nel primo caso, un'alta proporzione di tentativi rifiutati per eccessiva similarità.

4.3.3 Varietà stilistica

Il solo mascheramento non garantisce, di per sé, la varietà linguistica richiesta da O2: un modello lasciato libero di riformulare tende a convergere su un piccolo numero di strutture sintattiche ricorrenti. Il sistema affronta questo problema definendo un insieme di dieci registri stilistici predefiniti (ad esempio una formulazione tecnica in stile interrogazione a database, una clinico-formale, una colloquiale, una statistica orientata al conteggio), esplicitamente descritti al modello a ogni chiamata.

Gli stili non vengono assegnati alle varianti in modo casuale, ma percorsi in sequenza ciclica, in modo che la distribuzione degli stili sulle varianti generate risulti uniforme anziché soggetta alla variabilità di un'estrazione casuale. Per ciascuno stile, il sistema mantiene inoltre una memoria a breve termine delle formulazioni e delle parole di apertura usate più di recente: un nuovo candidato che ripropone un'apertura già usata di recente per lo stesso stile viene rifiutato dalla validazione allo stesso modo di una riformulazione troppo simile a livello di intero testo, estendendo così il controllo di varietà anche alla struttura delle prime parole della frase, non solo al suo contenuto complessivo.

Il sistema conserva infine, per ciascun record, anche una modalità alternativa priva di mascheramento, che riproduce il comportamento originario a chiamata singola sull'intero testo. Questa modalità non costituisce un secondo percorso di produzione, ma un termine di paragone esplicito, i cui risultati sono discussi assieme a quelli della modalità con mascheramento nel capitolo di valutazione.

Capitolo 5

Implementazione

In questo capitolo viene descritto come sono state implementate le varie parti del contributo di questa tesi, in maniera coerente con la progettazione svolta nel capitolo 4. La trattazione segue l'ordine cronologico in cui i due stadi del sistema sono stati sviluppati.

5.1 Implementazione del generatore

5.1.1 Architettura a mixin

La pipeline di generazione appena introdotta a livello di design (sezione 4.2.1) è realizzata concretamente da un'unica classe, `BulkDatasetGenerator`, ottenuta mediante ereditarietà multipla da sei *mixin*¹, uno per ciascuna fase della pipeline. Questa struttura permette di mantenere la corrispondenza diretta tra fase concettuale e componente software già motivata in fase di design, con un'unica interfaccia per l'intero processo di generazione: le diverse fasi possono infatti essere modificate o ampliate intervenendo principalmente sul mixin che ne implementa la relativa responsabilità, senza dover toccare gli altri. La composizione della classe principale è riportata nel listato 5.1.

Listing 5.1: Composizione della classe principale del generatore tramite mixin.

¹Un mixin è una classe pensata per essere combinata con altre tramite ereditarietà multipla, fornendo un insieme coeso di funzionalità senza costituire di per sé una gerarchia di tipi completa.

```
1 class BulkDatasetGenerator(  
2     FilterApiMixin,  
3     DiscoveryMixin,  
4     SkeletonMixin,  
5     PlanningMixin,  
6     ProgressMixin,  
7     OutputMixin,  
8 ):  
9     # parameters
```

Ciascun mixin realizza una o più delle fasi appena introdotte a livello di design, incapsulando le chiamate di rete o coordinando la logica di più fasi quando queste sono strettamente collegate tra loro:

- **FilterApiMixin** incapsula le chiamate di rete verso l'API interna del CVL;
- **DiscoveryMixin** costruisce la mappa del grafo dello schema;
- **SkeletonMixin** costruisce lo scheletro di una singola query;
- **PlanningMixin** determina quali combinazioni di proprietà devono essere campionate;
- **OutputMixin** esegue i controlli di qualità finali e gestisce la scrittura dei risultati su disco;
- **ProgressMixin** gestisce gli aspetti legati all'interazione con l'interfaccia Streamlit, tra cui l'aggiornamento della barra di avanzamento e la possibilità di interrompere l'esecuzione.

Le responsabilità non sono quindi indipendenti, ma cooperano secondo il flusso della generazione, ad eccezione di **ProgressMixin**, che costituisce invece una responsabilità trasversale, poiché può intervenire durante l'esecuzione delle diverse fasi senza rappresentare una fase autonoma della pipeline.

La separazione è particolarmente evidente nella gestione dell'interruzione della generazione (gestita proprio da **ProgressMixin**). Il metodo `_is_aborted()`, riportato nel listato 5.2, viene richiamato periodicamente durante la generazione per verificare se l'esecuzione debba essere interrotta.

Listing 5.2: Controllo dell'interruzione durante la generazione.

```
1 def _is_aborted(self) -> bool:
2     if self._abort_event.is_set():
3         return True
4     if not self.config.get("abortable", True):
5         return False
6     if self._can_paint() and bool(
7         st.session_state.get("abort_generation", False)
8     ):
9         self._abort_event.set()
10    return self._abort_event.is_set()
```

Il controllo combina due meccanismi. Il primo è l'evento di sincronizzazione interno `_abort_event`, che rappresenta lo stato di interruzione del run. Il secondo è lo stato dell'interfaccia Streamlit, dal quale viene letto il valore di `session_state["abort_generation"]`. Quando l'utente richiede l'interruzione, il metodo imposta l'evento interno; le chiamate successive possono quindi rilevare direttamente tale stato senza dover interrogare nuovamente l'interfaccia.

Il controllo dello stato di Streamlit viene eseguito soltanto quando l'esecuzione è configurata come interrompibile, tramite `config["abortable"]`, e quando il contesto consente di interagire con l'interfaccia. In questo modo la stessa implementazione può essere utilizzata anche nei contesti che non espongono un controllo di interruzione, come le esecuzioni da riga di comando o un pannello di generazione batch.

Le costanti di classe di `BulkDatasetGenerator`, riportate nel listato 5.3, definiscono alcuni parametri di funzionamento condivisi dalle diverse fasi della generazione.

Listing 5.3: Costanti di classe utilizzate dal generatore.

```
1 class BulkDatasetGenerator(
2     # parameters
3 ):
4     BUFFER_SIZE = 50
5     MAX_DISCOVERY_WORKERS = 8
6     GENERATION_WORKERS = 4
7     DISCARD_SAMPLES = 500
8     VALUE_REROLLS = 4
9     MAX_STORED_VALUES = 5000
10    SAMPLING_GIVEUP_STREAK = 250
```

In particolare, tali parametri controllano la dimensione del buffer di scrittura, il grado di parallelismo utilizzato durante la scoperta e la generazione, il numero massimo di valori conservati per proprietà e la soglia di tentativi consecutivi senza successo oltre la quale il campionamento viene interrotto. A questi si aggiungono i parametri relativi alla resilienza delle chiamate di rete, costituiti da un timeout di base di 15 secondi, fino a due ripetizioni in caso di fallimento e un secondo di attesa tra un tentativo e l'altro.

5.1.2 Scoperta della mappa del grafo

Prima di generare qualunque record, il sistema costruisce un'unica mappa completa delle proprietà e delle relazioni disponibili per ciascuna entità dello schema, interrogando l'API interna della piattaforma CVL. Questa mappa viene costruita una sola volta e riutilizzata per l'intera esecuzione, evitando di ripetere le stesse chiamate di rete ogni volta che un'entità già scoperta viene esplorata di nuovo. Il dettaglio implementativo di come l'API venga interrogata non è discusso in questa sede, trattandosi di un'interfaccia della piattaforma CVL e non di un contributo originale di questa tesi; qui rileva solo il risultato che il generatore ne ricava, riportato in forma semplificata nel listato 5.4: per ogni entità, l'insieme delle sue proprietà filtrabili (`properties`) e delle entità verso cui può proseguire una relazione (`relations`).

Listing 5.4: Struttura semplificata della mappa del grafo, ridotta a titolo esemplificativo a una sola proprietà e una sola relazione dell'entità Patient.

```
1 graph_map = { "properties": { "Patient": [ { "predicate": {"value":  
2   "has_gender", "label": "gender"}, "datatype": "string", "n_values": 2,  
3   "values": [ { "value": "F", "label": "Female"}, { "value": "M", "label":  
4   "Male"}, ], }, ], }, "relations": { "Patient": [ { "predicate": {"value":  
5   "has_diagnosis", "label": "diagnosis"}, "target": {"value": "Diagnosis",  
6   "label": "Diagnosis"} }, ], }, }
```

Ogni proprietà filtrabile porta con sé, oltre al predicato che la identifica, il tipo di dato, il numero di valori distinti disponibili (`n_values`) e i valori stessi, già raccolti in questa fase (fino al tetto `MAX_STORED_VALUES` descritto più sotto); ogni relazione porta invece il predicato usato per il salto e l'entità di destinazione (`target`).

La scelta di indicizzare entrambe le sezioni per nome di entità, invece di usare una singola lista di record, non è casuale. La mappa viene infatti interrogata ripetutamente durante la scoperta delle route (sezione 5.1.3), una volta per ciascuna entità di volta in volta esplorata, e un dizionario permette di recuperare le proprietà o le relazioni di una data entità in tempo costante, invece di dover scorrere l'intera struttura ogni volta.

Costruire la mappa una volta sola, prima di iniziare qualunque generazione, comporta anche un compromesso più ampio. Quando viene aperta, ogni entità viene interrogata come punto di partenza isolato, con una storia di selezione vuota, indipendentemente dal fatto che nell'esplorazione vera delle route quella stessa entità venga poi raggiunta attraverso un percorso diretto o attraverso più salti relazionali. La mappa assume quindi che le proprietà e le relazioni disponibili per un'entità dipendano solo dal suo tipo, non dal percorso concreto con cui la si raggiunge, a differenza dell'API del CVL, che può restituire relazioni diverse a seconda della storia di selezione accumulata fino a quel punto. Questo costo viene accettato deliberatamente, perché l'alternativa, interrogare l'API da capo ogni volta che una qualunque entità viene esplorata, è proprio il collo di bottiglia che si vuole evitare.

Per ridurre il tempo complessivo di questa fase, le interrogazioni relative alle diverse opzioni di una stessa entità vengono eseguite in parallelo, fino a un massimo configurabile per volta (`MAX_DISCOVERY_WORKERS`), tramite un pool di thread; i valori restituiti per ciascuna proprietà filtrabile vengono conservati fino a un tetto massimo (`MAX_STORED_VALUES`), per non tenere in memoria un elenco potenzialmente enorme quando un campo ammette moltissimi valori distinti.

5.1.3 Costruzione delle route

Con la mappa disponibile, per ciascuna entità radice richiesta, si esplora tramite una visita in profondità (DFS) ogni combinazione raggiungibile di salti relazionali fino a un limite di profondità configurato.

Listing 5.5: Visita in profondità della mappa del grafo per la costruzione delle route disponibili a partire da un'entità radice.

```
1 || def dfs(type_value, chain, relations_used, visited, ancestor_terminals):
```

```
2     terminals = [ {"predicate": p["predicate"], "chain_prefix": list(chain),
3                   ...} for p in properties.get(type_value, []) ] if terminals: routes.append({
4     "chain": list(chain), "terminals": ancestor_terminals + [{"**t,
5     "is_final_entity": True} for t in terminals], })
6
7     if relations_used >= max_relations: return inherited = ancestor_terminals +
8     [{"**t, "is_final_entity": False} for t in terminals] for rel in
9     relations.get(type_value, []): target_value = (rel.get("target") or
10    {}).get("value") if not target_value or target_value in visited:
11    continue dfs( target_value, [*chain, {"predicate": rel["predicate"],
12    "reference_to": rel.get("target")}], relations_used + 1, visited |
13    {target_value}, inherited, )
```

Ogni volta che la visita raggiunge un nodo con proprietà filtrabili, viene registrata una *route*, ovvero l'insieme di:

- la sequenza di salti relazionali per raggiungerlo (la *chain*);
- l'insieme delle proprietà disponibili in quel punto (i *terminal*), che include sia quelle del nodo corrente sia quelle ereditate dai nodi attraversati in precedenza.

La distinzione dei terminal in proprietà ereditate e non (tramite variabile booleana) garantisce, nel passo successivo di campionamento, che ogni combinazione di proprietà scelta includa almeno un vincolo sul nodo in cui la route si ferma davvero, evitando di costruire query che vincolano solo nodi di passaggio.

Sebbene la visita che le genera sia intrinsecamente ad albero, l'insieme delle route risultanti viene appiattito in una singola lista di record indipendenti per facilitare la fase di campionamento (sezione 5.1.4), la quale deve poter scorrere ed estrarre a caso qualunque route tra tutte quelle disponibili, con accesso diretto e costante a ciascuna, senza dover ripercorrere un albero ogni volta per ricostruire chain e terminal di un nodo scelto casualmente al suo interno.

5.1.4 Pianificazione: stima combinatoria e campionamento

Prima di generare i record, il sistema calcola quante coppie prompt-query distinte siano effettivamente raggiungibili con le route disponibili e i vincoli configurati, in modo da non richiedere più record di quanti lo spazio combinatorio possa realmente offrire; solo dopo, campiona a caso tra le combinazioni così stimate per costruire

ogni singolo record. I due passi sono di natura diversa: la stima è un conteggio puramente matematico, espresso di seguito con delle formule; il campionamento è invece un algoritmo vero e proprio, mostrato più avanti in codice.

Stima combinatoria

Sia r una route, T_r l'insieme dei suoi terminal e $T_r^f \subseteq T_r$ i terminal finali (quelli del nodo dove la route si ferma davvero). Sia $v(t)$ il numero di combinazioni di valori di un terminal, e $\mathcal{C}_k(T_r)$ le combinazioni di k terminal di T_r (nel senso standard, $\binom{|T_r|}{k}$ possibilità) che includono almeno un terminal di T_r^f . Con $p_{\min}$, $p_{\max}$ i limiti *min_properties*/*max_properties* configurati, il peso di r è

$$w(r) = \begin{cases} 0 & \text{se } T_r^f = \emptyset \\ \sum_{k=p_{\min}}^{\min(p_{\max}, |T_r|)} \sum_{c \in \mathcal{C}_k(T_r)} \prod_{t \in c} v(t) & \text{altrimenti.} \end{cases} \quad (5.1)$$

Un record spesso combina più route insieme. Sia V l'insieme delle route con peso non nullo, e $\text{indip}(S)$ vera quando nessuna coppia di route in S ha una chain prefisso dell'altra (sezione 5.1.3). Con $s_{\min}$, $s_{\max}$ i limiti *min_paths*/*max_paths* configurati, il tetto teorico dell'entità è

$$N = \sum_{s=s_{\min}}^{s_{\max}} \sum_{\substack{S \subseteq V \\ |S|=s \\ \text{indip}(S)}} \prod_{r \in S} w(r). \quad (5.2)$$

È da sottolineare che N è di fatto una sovrastima, in quanto route indipendenti che condividono un prefisso condividono anche parte dei terminal, mentre il prodotto dei pesi utilizzato nella nostra formula le tratta come scelte separate. Per l'uso che se ne fa, cioè riconoscere quando lo spazio delle combinazioni è troppo piccolo rispetto ai record richiesti, un limite superiore basta, perciò possiamo evitare di complicare ulteriormente il calcolo. Proprio per evitare un'ulteriore inefficienza, l'algoritmo imposta una soglia di 500 000 combinazioni da esaminare, dopo la quale il conteggio esatto viene abbandonato e il campionamento procede senza un tetto noto in anticipo.

Campionamento

Per ogni record da generare, il numero di route da combinare viene scelto a caso entro i limiti configurati. Le route disponibili vengono poi estratte da un elenco mescolato in ordine casuale, scartando quelle che risultano *in conflitto* con una già scelta, cioè la cui chain è un prefisso di un'altra già selezionata, il che indicherebbe due vincoli ridondanti sullo stesso tratto di percorso. Il campionamento continua finché non si raggiunge il numero di record stimato dal tetto teorico (equazione (5.2)), oppure finché 250 tentativi consecutivi (`SAMPLING_GIVEUP_STREAK`) non producono nulla di nuovo, segno che lo spazio raggiungibile è stato esaurito.

Listing 5.6: Campionamento casuale di un insieme di route indipendenti tra loro per un singolo record.

```
1 def sample_independent_combo(target_p): pool = list(valid) rng.shuffle(pool)
2   chosen, chosen_keys = [], [] for r in pool: if len(chosen) >= target_p:
3     break key = chain_key_of[id(r)] if not routes_mod.conflicts(chosen_keys,
4     key): chosen.append(r) chosen_keys.append(key) return tuple(chosen)
```

Questa modalità di campionamento casuale (`_run_sampled`) non è l'unica prevista dal codice, esiste infatti anche una modalità a forza bruta (`_run_brute_force`) che enumera esaustivamente ogni combinazione possibile di route e proprietà. Quest'ultima modalità risulta utile quando l'obiettivo è ottenere una copertura completa dello spazio combinatorio; tuttavia non è stata utilizzata nei run di produzione, ma è stata comunque impiegata in fase di sviluppo per individuare eventuali errori di generazione, proprio grazie alla sua capacità di esplorare lo spazio delle combinazioni senza lasciarne fuori nessuna. Nella casistica affrontata da questa tesi però il dataset generato è destinato al fine-tuning, per il quale non è necessario enumerare tutte le combinazioni possibili, anzi è preferibile costruire un insieme sufficientemente ampio e vario di esempi rappresentativi, evitando al contempo di produrre una quantità di dati eccessiva rispetto allo scopo.

5.1.5 Esecuzione sequenziale e parallela

Una volta pianificata una combinazione di route e proprietà da trasformare in un record, ha inizio l'intero processo di costruzione del record vero e proprio, il quale comporta diverse chiamate di rete verso l'API del CVL. Questo processo

può essere eseguito in sequenza, una combinazione alla volta, oppure in parallelo su più thread, ciascuno impegnato a costruire un record diverso in contemporanea agli altri, a seconda della configurazione.

Listing 5.7: Scelta tra esecuzione sequenziale e parallela in base al numero di worker configurati.

```
1 workers = max(1, int(self.config.get("num_workers") or self.GENERATION_WORKERS))
2 if workers <= 1: self._run_jobs_sequential(jobs_iter, total_pairs, p, target,
3 max_no_new) return
```

Se il numero di worker configurato è maggiore di uno, le combinazioni campionate vengono sottomesse a un pool di thread, fino a `GENERATION_WORKERS` worker attivi contemporaneamente (di default 4), ciascuno impegnato nella costruzione di un record indipendente. Il sistema non sottomette tuttavia tutte le combinazioni disponibili in un'unica operazione, poiché ciò potrebbe determinare l'accumulo in coda di un numero molto elevato di job in attesa. Viene invece mantenuta una finestra di lavori sottomessi ma non ancora completati, di dimensione pari al doppio del numero di worker attivi; quando la generazione di un record termina e il risultato viene salvato, il relativo spazio viene immediatamente riutilizzato per sottomettere una nuova combinazione.

Questa strategia consente, in condizioni di parallelismo effettivo, di mantenere i worker occupati riducendo i tempi morti, senza introdurre una quantità di lavoro in coda eccessiva rispetto alla capacità di elaborazione disponibile. Il meccanismo è stato predisposto principalmente in vista di future esecuzioni su larga scala, nelle quali il parallelismo potrebbe ridurre significativamente il tempo complessivo di generazione.

Durante la produzione del dataset trattato, il numero di worker è stato invece impostato esplicitamente a 1 (listato 5.8) eseguendo così la generazione in modalità single-threaded. La scelta è dettata dal collo di bottiglia rappresentato dalle chiamate all'API, le cui richieste concorrenti non avrebbero ridotto i tempi in proporzione, anzi avrebbero rischiato di comprometterne stabilità e prestazioni. Il parallelismo è rimasto invece attivo nella scoperta della mappa (sezione 5.1.2), dove il carico delle operazioni si presta meglio all'esecuzione concorrente.

Proprio perché il parallelismo può comunque essere attivato lo stato condiviso tra i thread non è protetto da un unico lock globale ma da quattro lock distinti,

ciascuno a guardia di una porzione di stato diversa e indipendente dalle altre: un lock protegge gli insiemi usati per riconoscere i duplicati (sezione 5.1.9), un altro la cache dello scheletro (sezione 5.1.6), un altro ancora le statistiche interne sulle chiamate effettuate, e l'ultimo il registro dei job attualmente in esecuzione. Tenerli separati, invece di usarne uno solo per tutto, evita che un thread resti bloccato in attesa di una struttura dati che non gli serve, solo perché un altro thread ne sta aggiornando una diversa nello stesso istante.

5.1.6 Costruzione dello scheletro: percorso online e offline

Ogni combinazione campionata deve essere tradotta in una struttura dati intermedia, chiamata *scheletro*, che rappresenta i nodi del grafo coinvolti nella query, prima ancora che i valori concreti vengano scelti. Il sistema offre due percorsi alternativi per costruirla, insieme ad altre opzioni di comportamento, tutte raccolte in un'unica configurazione condivisa da entrambe le liste di tier.

Le opzioni condivise

Listing 5.8: Configurazione condivisa usata nei run di produzione, per entrambe le liste di tier.

```
1 COMMON_CONFIG = { "num_workers": 1, "offline_history": True,
2   "include_value_spans": True, "skip_navigation_in_prompt": False,
3   "validate_sparql": True, }
```

Ciascuna di queste cinque opzioni governa un aspetto distinto del comportamento del generatore.

- `num_workers` a 1 è la scelta, già motivata in sezione 5.1.5, di eseguire la generazione in sequenza anziché in parallelo.
- `include_value_spans` controlla se il generatore debba tracciare le posizioni dei valori letterali nel prompt (sezione 5.1.8, dove è ripreso in dettaglio).
- `skip_navigation_in_prompt`, se attivato ometterebbe dal testo del prompt le clausole di sola navigazione, rendendolo più corto ma meno esplicito sul percorso nel grafo; è stato tenuto disattivato in produzione per non perdere quell'informazione.

- `validate_sparql` controlla se la query SPARQL restituita dall'API debba passare un controllo di sintassi prima di essere accettata; sarà ripreso più avanti insieme agli altri controlli di qualità (sezione 5.1.9).
- `offline_history` decide se lo scheletro venga ricostruito in locale oppure costruito camminando dal vivo dentro l'API; riguarda direttamente le due sottosezioni seguenti.

Il percorso online

Quando `offline_history` è disattivato, lo scheletro viene costruito camminando passo per passo tramite chiamate ricorsive nell'API del CVL, esplorando in modo consequenziale proprietà e relazioni. Per ogni terminal della combinazione, il sistema percorre uno per uno i salti della sua chain (sezione 5.1.3) e, a partire dall'entità radice, a ogni salto chiede all'API di proseguire dal nodo raggiunto fino a quel momento verso il nodo successivo, accumulando progressivamente lo stato della query in costruzione finché non raggiunge il nodo del terminal stesso. Questa procedura ripercorre esattamente il percorso che la fase di scoperta della mappa aveva già seguito per costruire la mappa iniziale (sezione 5.1.2); la differenza è che lì veniva fatto una volta per ogni proprietà isolata, per costruire la mappa, mentre qui viene ripetuto per intero a ogni singolo record. Durante questa fase di costruzione online sono inoltre presenti alcuni meccanismi di resilienza, introdotti per gestire le condizioni anomale e gli eventuali problemi che possono verificarsi durante le chiamate all'API. Tali meccanismi si sono rivelati necessari nel corso dello sviluppo e vengono descritti di seguito.

- se l'API impiega troppo a rispondere, il fallimento è considerato transitorio, perché un tentativo successivo potrebbe ancora avere successo, e non viene memorizzato in nessuna cache;
- se invece risponde ma senza risultati utili, il terminal è considerato irraggiungibile in modo permanente, con due conseguenze distinte:
 - viene memorizzato in una cache condivisa così che i tentativi successivi lo scartino subito senza ripetere l'esplorazione;

- il record in corso esclude dalla combinazione quel singolo terminal e la generazione prosegue con gli altri rimasti (esiste una modalità più rigida, mai usata in produzione, che interromperebbe l'intero tentativo al primo terminal irraggiungibile);
- indipendentemente dai primi due, per non ripetere l'intera esplorazione quando la stessa combinazione di terminal ricorre in record diversi, lo scheletro completo viene tenuto in una cache separata, indicizzata sulla combinazione esatta richiesta, e riusato direttamente se si ripresenta, con solo il riempimento dei valori concreti (sezione 5.1.7) che cambia da un record all'altro.

Il percorso offline

Con `offline_history` attivo, come nei run di produzione, tutta la camminata appena descritta viene saltata e lo scheletro non si costruisce esplorando dal vivo l'API del CVL, ma viene ricostruito interamente in locale a partire dai dati che la scoperta della mappa aveva già raccolto per ciascuna proprietà (sezione 5.1.2). Per ogni terminal coinvolto nella combinazione, il sistema ricrea la stessa sequenza di nodi intermedi che un'esplorazione dal vivo avrebbe prodotto, camminando sulla sequenza di salti già nota (registrata al momento della scoperta delle route, sezione 5.1.3) e usando i valori possibili già memorizzati per quel terminal, senza doverli richiedere di nuovo; con lo scheletro pronto, si arriva così direttamente all'unico passaggio che nessuno dei due percorsi può evitare, ovvero la chiamata finale descritta di seguito.

Listing 5.9: Lo scheletro appena ricostruito per i terminal `gender` e `birth date`, ancora privo di valori.

```
1 history = [ { "query_nodes": [ { "predicate": { "label": "gender" }, "values":  
2     None, "value_range": None }, { "predicate": { "label": "birth date" }, "values":  
3     None, "value_range": None }, ] } ]
```

Indipendentemente dal percorso scelto per lo scheletro, il testo effettivo della query SPARQL viene sempre ottenuto con un'unica chiamata finale all'API, che traduce la struttura accumulata in una query eseguibile. Questo passaggio non è mai eseguito localmente, in nessuna delle due modalità, rappresentando così l'unica

chiamata API fatta dal percorso offline. Su questa stessa chiamata finale interviene anche il quinto flag di listato 5.8, `validate_sparql`, ripreso e motivato più avanti insieme agli altri controlli di qualità applicati al risultato finale (sezione 5.1.9).

5.1.7 Riempimento dei valori

Una volta pronto lo scheletro, ogni suo nodo vuoto viene riempito con valori concreti scelti a caso dal pool associato al terminal corrispondente.

Listing 5.10: Riempimento di un nodo dello scheletro con valori concreti, distinguendo tra intervalli e insiemi discreti.

```
1 def fill_node(node, term, pool, min_values, max_values): k =
2     random.randint(min_values, min(max_values, len(pool))) chosen =
3     random.sample(pool, k) if _is_range_datatype(term.get("datatype")): ordered
4     = sorted(chosen, key=lambda v: v["value"]) node["value_range"] = {"min":
5     ordered[0]["value"], "max": ordered[-1]["value"]} else: node["values"] =
6     chosen
```

Il numero di valori da usare per un dato vincolo è scelto a caso entro i limiti configurati (listato 5.10), evitando però di selezionare l'intero dominio possibile di un campo, perché un vincolo che copre tutti i valori ammessi non escluderebbe di fatto nessun paziente, risultando così privo di potere selettivo. Se il campo è una data o un valore numerico, i valori scelti vengono usati per costruire un intervallo anziché un elenco discreto.

Listing 5.11: Lo stesso scheletro di listato 5.9, dopo il riempimento: elenco discreto per il campo categoriale, intervallo per quello di tipo data.

```
1 history = [ { "query_nodes": [ {"predicate": {"label": "gender"}, "values":
2     [{"value": "F", "label": "Female"}], "value_range": None}, {"predicate":
3     {"label": "birth date"}, "values": None, "value_range": {"min":
4     "2016-01-01", "max": "2019-01-01"}}, ] } ]
```

5.1.8 Generazione del prompt e tracciamento degli span

Definiamo un segmento come un pezzo di testo accompagnato da un booleano che indica se il testo contenga un valore letterale, come una data o un codice, oppure solo una parola di struttura della frase come *with* o *and*. Con lo scheletro completo di valori, il prompt viene costruito ri assemblando i suoi nodi in una lista

di segmenti, concatenati uno via l'altro formando il prompt finale. La formulazione di ciascun segmento dipende da cosa contiene il nodo di origine:

- se il nodo ha valori scelti (sezione 5.1.7), il segmento inizia con *with* seguito dai valori stessi, uniti da *or* se sono due, o da una serie di virgole con un *or* finale se sono di più;
- se rappresenta un intervallo, diventa *between X and Y*;
- un nodo senza valori propri, che serve solo a passare al nodo successivo della chain senza introdurre un vincolo, produce invece un segmento con *which* al posto di *with*; un nodo *diagnosis* senza valori seguito da un nodo *stage* con valore *II* produce ad esempio *"which diagnosis with stage II"*.

I segmenti di uno stesso passo della chain vengono uniti con un semplice *and*, e lo stesso connettivo lega tra loro i passi successivi, a partire da un'apertura fissa (*"Find all <entità>"*).

Listing 5.12: Costruzione del prompt segmento per segmento, con tracciamento diretto della posizione di ciascun valore letterale.

```
1 def build_prompt_with_spans(root_label, history, skip_navigation=False):
2     segments = _prompt_segments(root_label, history, skip_navigation) pieces,
3     spans = [], [] for is_value, text in segments: if is_value: base =
4     sum(len(p) for p in pieces) spans.append((base, base + len(text)))
5     pieces.append(text) return "".join(pieces), spans
```

Mentre i segmenti vengono concatenati uno alla volta, la posizione di ciascun valore inserito nel testo viene registrata. Determinare gli intervalli solo a partire dal prompt finale sarebbe infatti ambiguo nel caso in cui lo stesso valore comparisse, per coincidenza, anche in un'altra posizione della frase. Registrare le coordinate durante la costruzione del testo elimina quindi questa ambiguità senza richiedere controlli aggiuntivi. La funzione restituisce sempre la coppia (**prompt**, **value_spans**) coerente tra loro, qualunque sia il testo prodotto; solo più avanti, al momento di salvare il record completo, il flag **include_value_spans** (sezione 5.1.6) decide se il campo **value_spans** venga scritto nell'output finale o scartato, sempre attivo nei run di produzione.

Applicata allo scheletro pieno di listato 5.11, la procedura produce il prompt *"Find all patient with gender Female and birth date between 2016-01-01 and 2019-01-01"*, con `value_spans` pari a `[(29, 35), (59, 69), (74, 84)]`, che corrispondono rispettivamente a *"Female"*, *"2016-01-01"* e *"2019-01-01"*.

5.1.9 Controlli di qualità e deduplicazione

Prima di essere salvata, ogni coppia prompt-query attraversa tre controlli in sequenza.

1. Il primo verifica che la query restituita dall'API contenga davvero una clausola `FILTER` o `VALUES`, oppure almeno uno dei valori letterali scelti. Questo controllo esiste perché l'API potrebbe, in linea di principio, restituire una query sintatticamente valida ma priva di ogni vincolo reale, per esempio se un valore non venisse tradotto correttamente in un filtro effettivo: una query del genere risulterebbe innocua da eseguire, ma completamente slegata dal prompt che dovrebbe rappresentare, e finirebbe nel dataset come esempio scorretto senza che nulla lo segnali. Se nessuna delle due condizioni è soddisfatta, il record viene scartato con il motivo esplicito *"the backend ignored our filters"*.

Listing 5.13: Il primo controllo: verifica che i vincoli scelti siano effettivamente presenti nella query SPARQL restituita.

```
1 def _filters_landed(sparql, chosen_values): if "FILTER" in sparql or "VALUES"
    " in
2     sparql: return True return any(v in sparql for v in chosen_values)
```

2. Il secondo verifica la validità sintattica della query tramite un vero parser SPARQL, la libreria `rdflib`². Serve a intercettare query malformate prima che finiscano nel dataset: un esempio di addestramento con una query che non si può nemmeno interpretare sarebbe peggio che inutile, perché insegnerebbe al modello una sintassi scorretta. Il controllo è condizionato dal parametro `validate_sparql` (listato 5.8), impostato a vero nei run di pro-

²<https://rdflib.readthedocs.io/>

duzione; ogni query che non superava il parsing veniva quindi scartata per davvero, con il motivo esplicito *"the SPARQL does not parse (rdflib)"*.

Listing 5.14: Il secondo controllo: il parsing sintattico tramite `rdflib` è condizionato dal parametro di configurazione.

```
1 syntax_error = ( validate.sparql_error(sparql) if
2   self.config.get("validate_sparql", True) else None )
```

3. Il terzo verifica che la coppia non sia un duplicato, tenendo in memoria l'insieme dei prompt già visti e l'insieme degli hash calcolati su ogni coppia prompt-query già accettata: un nuovo prompt viene scartato se è già in quell'insieme, una nuova coppia se il suo hash coincide con uno già registrato. Senza questo controllo, il campionamento potrebbe scegliere per caso, in record diversi, la stessa combinazione di route e valori, riducendo la varietà effettiva senza che il conteggio totale dei record se ne accorga. In caso di prompt duplicato, il sistema ritenta con nuovi valori scelti a caso fino a 4 volte (`VALUE_REROLLS`) prima di arrendersi.

Solo le coppie che superano tutti e tre i controlli vengono accumulate in un buffer e scritte su disco a gruppi di 50 (`BUFFER_SIZE`).

5.1.10 Generazione manuale a entità singola

Tutto quanto descritto finora in questo capitolo, dalla scoperta della mappa fino ai controlli di qualità, produce record per *una* entità alla volta, data una combinazione specifica di route e proprietà. Il modo più diretto per eseguire questo motore è farlo a mano, entità per entità, tramite un pannello dell'interfaccia Streamlit, organizzato in cinque sezioni.

- **Graph Map:** preconditione per tutto il resto, ha un pulsante che permette di caricare la mappa del grafo (sezione 5.1.2) e ne mostra un riepilogo (numero di entità, proprietà e relazioni scoperte) una volta completata.
- **Mode:** sceglie tra questa modalità a entità singola e la generazione automatica da file di tier (sezione 5.1.11).

- **Starting Entity:** un menu a tendina per scegliere l'unica entità radice da generare.
- **Configuration:** quattro slider, uno per relazioni da attraversare, proprietà per percorso, percorsi simultanei e valori per proprietà.
- **Generation Mode e History building:** espongono direttamente, tramite interruttori e caselle di spunta, parametri già discussi altrove in questo capitolo, come la scelta tra campionamento e forza bruta e il numero di worker (sezioni 5.1.4 e 5.1.5), o i flag `offline_history`, `include_value_spans`, `skip_navigation_in_prompt` (sezione 5.1.6).

Gli estremi dei quattro slider di configurazione non sono fissi, ma vengono calcolati al volo scoprendo, tramite `discover_routes_from_map` (sezione 5.1.3), quante route siano effettivamente raggiungibili da quell'entità con i limiti scelti.

Questa modalità non è pensata per costruire un dataset completo, ma per esplorare e calibrare. Al termine della generazione, infatti, il pannello non si limita a riportare un conteggio finale, ma mostra due schede separate: una con i record effettivamente creati, in un'anteprima tabellare di prompt e query, e una con quelli scartati, raggruppati per motivo di scarto esatto, tra cui i tre già discussi in sezione 5.1.9 (filtri non atterrati nella query, query sintatticamente non valida, duplicato), con la possibilità di filtrare e ispezionare i singoli tentativi falliti per una singola categoria. È proprio questo riscontro dettagliato a rendere la modalità manuale utile a decidere quali parametri applicare a un tier con decine di entità e tempi di generazione elevati, dove un errore nei parametri rischierebbe di sprecare risorse di calcolo e prolungare inutilmente i tempi di attesa.

5.1.11 Configurazione dei tier da file esterno

La procedura manuale appena descritta agisce su una sola entità alla volta, ma entità diverse dello schema possono richiedere impostazioni diverse tra loro a seconda di quanto siano rilevanti per il compito, di quanto siano profondamente connesse al resto del grafo, o di quante variazioni valga la pena rappresentarne.

Il generatore permette di raggruppare più entità sotto uno stesso insieme di parametri condivisi, un *tier*, così da poter personalizzare queste impostazioni gruppo

per gruppo invece di doverle applicare in modo uniforme a tutto lo schema, e di poter riutilizzare la stessa configurazione, o adattarla facilmente, in generazioni future. Per ciascun tier configurato, per ciascuna entità elencata al suo interno, viene eseguito lo stesso motore di generazione già descritto nelle sottosezioni precedenti, con i parametri propri di quel tier (gli stessi utilizzati nella sezione 5.1.10) e un numero massimo di record da generare per quell'entità (`sample_size_per_entity`). I record delle singole entità confluiscono infine in un unico file (sezione 5.1.12).

Questa configurazione esterna è esposta, sul lato dell'interfaccia Streamlit, tramite un pannello dedicato che permette di caricare il file e ne mostra a schermo un riepilogo, con il numero di gruppi e di entità coinvolte, prima di abilitare il pulsante di avvio; l'avanzamento della generazione viene poi tradotto in una barra di progresso calcolata sul numero di entità completate rispetto al totale. L'output finale viene poi reso disponibile direttamente come file scaricabile al termine del processo, eliminando così la necessità di un qualunque accesso al filesystem del server per chi utilizza lo strumento.

La configurazione dei tier sfrutta una funzione di validazione che non solo controlla che ogni gruppo contenga tutte le chiavi richieste, ma anche la coerenza dei range di valori selezionati e l'unicità dei nomi dei gruppi; un file scritto male viene così respinto subito, con un motivo preciso, invece di far fallire la generazione a metà.

5.1.12 Split del dataset

Una volta che tutte le entità dei tier configurati sono state processate e unite in un unico file, l'ultimo passo divide i record in train, validation e test.

Listing 5.15: Suddivisione del dataset in train, validation e test, stratificata per entità di origine.

```
1 for entity, recs in by_entity.items(): rng.shuffle(recs) n = len(recs) n_val =  
2     max(1, round(n * SPLIT_RATIOS["val"])) if n >= 10 else 0 n_test = max(1,  
3     round(n * SPLIT_RATIOS["test"])) if n >= 10 else 0 n_train = n - n_val -  
4     n_test splits["train"].extend(recs[:n_train])  
5     splits["val"].extend(recs[n_train:n_train + n_val])  
6     splits["test"].extend(recs[n_train + n_val:])
```

I record vengono raggruppati per entità di origine e suddivisi separatamente per ciascun gruppo, con proporzioni 80/10/10 e seme fisso a 42 per la riproducibilità (listato 5.15), garantendo così che ogni entità sia rappresentata nella stessa proporzione in tutti e tre gli split, invece di rischiare che un'entità poco frequente finisca per puro caso quasi interamente in uno solo di essi. Le entità con meno di 10 record totali non vengono suddivise e finiscono interamente in train, perché un campione così piccolo non permetterebbe una validazione o un test statisticamente significativi.

Lo split avviene prima dello stadio di augmentation, non dopo, per una ragione precisa. Se l'augmentation venisse eseguita sull'intero dataset ancora unito, più varianti parafrasate dello stesso record sorgente potrebbero finire sparse su split diversi, contaminando la separazione tra train e test con esempi troppo simili tra loro. Il generatore lo ricorda esplicitamente in un messaggio di log al termine dello split, indicando di eseguire la pipeline di augmentation separatamente su ciascuno dei tre file prodotti.

5.2 Implementazione dell'augmentation

La fase di *data augmentation* ha lo scopo di estendere la varietà linguistica dei prompt generati dal primo stadio della pipeline, garantendo al contempo che il significato semantico e i riferimenti puntuali alla query SPARQL sottostante rimangano inalterati (O2, RNF3).

Nelle prime fasi di esplorazione del progetto, la variazione stilistica è stata affidata direttamente a un modello linguistico, fornendo il testo in chiaro e richiedendone la rielaborazione in diversi registri linguistici (dallo stile clinico-formale a quello colloquiale). Questa impostazione ingenua ha tuttavia evidenziato severi limiti pratici:

1. **Alterazione e perdita di valori clinici (*hallucination*):** Il modello tendeva a modificare o omettere identificativi, date, codici clinici e valori booleani, compromettendo la corrispondenza univoca tra il prompt e i filtri della query SPARQL.

2. **Instabilità al crescere della complessità:** All'aumentare del numero di vincoli e valori presenti nella domanda originale, la qualità della parafrasi degradava rapidamente, portando a omissioni di intere clausole.
3. **Convergenza stilistica:** Lasciato libero, l'LLM tendeva a riproporre le medesime strutture sintattiche ricorrenti, vanificando l'obiettivo di varietà stilistica.

Per superare queste criticità e garantire un tasso di accettazione dei record economicamente sostenibile (RNF1), la pipeline di augmentation è stata completamente riprogettata attorno a un meccanismo rigoroso di **mascheramento dei valori letterali** (mediante placeholder opachi) associato a un ciclo iterativo di **generazione guidata, validazione e fallback**.

Nelle sottosezioni seguenti vengono descritte le componenti software che realizzano tale architettura.

5.2.1 Mascheramento

Il primo passo di ogni record è la sostituzione di ciascun valore letterale con un token opaco (listato 5.16).

Listing 5.16: Generazione dei placeholder e mascheramento del testo a partire dagli span di valore.

```
1 TOKEN_ALPHABET = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"
2
3 def generate_placeholder_tokens(n): tokens = set() while len(tokens) < n:
4     tokens.add("[VAL_" + "".join(random.choices(TOKEN_ALPHABET, k=4)) + "]")
5     return list(tokens)
6
7 def mask(text, spans): spans = sorted(spans) tokens =
8     generate_placeholder_tokens(len(spans)) mapping, pieces, cursor = {}, [], 0
9     for (start, end), placeholder in zip(spans, tokens):
10         pieces.append(text[cursor:start]) mapping[placeholder] = text[start:end]
11         pieces.append(placeholder) cursor = end pieces.append(text[cursor:]) return
12         "".join(pieces), mapping
```

L'alfabeto dei token esclude deliberatamente i caratteri facilmente confondibili tra loro a colpo d'occhio (I, L, O, 0, 1); i token vengono generati dentro un insieme (set) finché non se ne hanno abbastanza di univoci, escludendo così collisioni tra

due placeholder nello stesso record. La funzione `mask` opera direttamente sugli span di posizione prodotti dal generatore (sezione 5.1.8), tagliando il testo esattamente su quelle coordinate senza alcuna ricerca a posteriori. Due funzioni di regex ricorrono in tutto il modulo per riconoscere i placeholder, con scopi diversi: una versione rigida (`VAL_RE`), che combacia solo con il formato esatto appena mostrato, usata per estrarre i placeholder autentici da un testo; e una versione permissiva (`LOOSE_VAL_RE`), che riconosce anche forme storpiate o inventate, usata in fase di validazione per intercettare allucinazioni del modello (sezione 5.2.5).

5.2.2 Riformulazione booleana e suggerimenti naturali

Non tutti i placeholder attraversano l'intera pipeline di generazione: i valori booleani vengono eliminati subito, prima ancora di contattare il modello (listato 5.17).

Listing 5.17: Eliminazione anticipata dei placeholder booleani, sostituiti con una forma proposizionale.

```
1 def simplify_boolean_phrasing(text, mapping): new_mapping = dict(mapping) for
2     placeholder, value in mapping.items(): if value.lower() not in {"true",
3     "false"}: continue is_true = value.lower() == "true" repl = r"with \1" if
4     is_true else r"without \1" esc = re.escape(placeholder) stop =
5     r"(?:with|which|and)" which_re = re.compile(rf"\bwhich ((?:?!{stop}\b)\w+
6     ?){1,5}) is {esc}", re.IGNORECASE) with_re = re.compile(rf"\bwith
7     ((?:?!{stop}\b)\w+ ?){1,5}) {esc}", re.IGNORECASE) if
8     which_re.search(text): text = which_re.sub(repl, text) del
9     new_mapping[placeholder] elif with_re.search(text): text = with_re.sub(repl,
10    text) del new_mapping[placeholder] return text, new_mapping
```

Un placeholder che rappresenta un valore vero viene sostituito con *"with"* seguito dal termine clinico associato, uno falso con *"without"*; in entrambi i casi il placeholder viene rimosso anche dalla mappa restituita, non solo dal testo, così il resto della pipeline non si aspetta più di doverlo ritrovare. Per tutti gli altri valori mascherati, una funzione separata (`build_hint_map`) prepara un suggerimento in linguaggio naturale, in ordine di priorità: data ISO trasformata in forma scritta per esteso, numero piccolo (fino a 999) trasformato in lettere, etichetta composta con trattino ridotta alla sola parte specifica; solo se nessuna di queste regole deterministiche si applica, viene interrogato il modello per un sinonimo d'uso comune. Questo suggerimento viene usato, con una probabilità configurabile, al momento

del ripristino finale dei valori (sezione 5.2.5), e in modo differente nella modalità priva di mascheramento (sezione 5.2.8).

5.2.3 Suddivisione in chunk per record complessi

Un record con molti valori mascherati viene spezzato in più porzioni prima di essere sottoposto al modello, per evitare il degrado di qualità osservato empiricamente oltre una certa densità di placeholder in una singola chiamata.

Listing 5.18: Suddivisione del testo mascherato in chunk, rispettando il vincolo sugli intervalli *between...and...*

```
1 MAX_VALUES_PER_CHUNK = 12
2
3 def split_into_chunks(masked_text, mapping, max_values_per_chunk): clauses =
4     _protect_between(masked_text).split(CLAUSE_SEP) chunks = [[]] count_in_chunk
5     = 0 for clause in clauses: n_values = len(VAL_RE.findall(clause)) if
6     count_in_chunk > 0 and count_in_chunk + n_values > max_values_per_chunk:
7     chunks.append([]) count_in_chunk = 0 chunks[-1].append(clause)
8     count_in_chunk += n_values return [_restore_between(CLAUSE_SEP.join(c)) for
9     c in chunks if c]
```

Il testo viene diviso in clausole separate dalla congiunzione *"and"*, accumulate in un chunk finché non si supererebbe la soglia di 12 valori (`MAX_VALUES_PER_CHUNK`); un meccanismo dedicato (`_protect_between/_restore_between`) sostituisce temporaneamente l'*"and"* interno a un intervallo *"between X and Y"* con un segnaposto interno che non compare mai per caso in un testo inglese (il carattere NUL), impedendo che un intervallo venga spezzato a metà tra due chunk diversi. Ogni record con un numero di valori pari o inferiore alla soglia produce un solo chunk, e non attraversa quindi alcuna suddivisione: nel dataset supplementare da 1331 record generato con vincolo fisso a 4 proprietà per record (sezione 5.1.11), l'80,6% dei record (1073 su 1331) è rimasto in un solo chunk, mentre il restante 19,4% (258 record) è stato diviso in due; nessun record ha richiesto più di due chunk, nemmeno quello con più valori in assoluto (20 valori, divisi in due chunk da 11 e 9).

5.2.4 Generazione iterativa: stile, retry e fallback

Ogni chunk, e successivamente ogni variante completa, viene generato tramite un ciclo di tentativi con feedback esplicito sul motivo di un eventuale rifiuto e temperatura crescente (listato 5.19).

Listing 5.19: Ciclo di generazione e validazione per un singolo chunk, con fallback esplicito in caso di fallimento persistente.

```
1 def generate_piece(model, piece_text, piece_mapping, style_desc, max_attempts,
2     base_temp, max_temp, avoid_history, indent, label,
3     avoid_text=None, avoid_reason=None, as_fragment=False,
4     language="English"): total_elapsed = 0.0 for attempt in
5     range(max_attempts): temperature =
6     temperature_for_attempt(attempt, max_attempts, base_temp,
7     max_temp) candidate, elapsed = ( paraphrase_fragment(model,
8     piece_text, piece_mapping, temperature, language,
9     avoid_text, avoid_reason) if as_fragment else
10    paraphrase_one(model, piece_text, piece_mapping, style_desc,
11    temperature, language, avoid_text, avoid_reason,
12    avoid_history if attempt == 0 else None) ) total_elapsed +=
13    elapsed reason = validation_reason(candidate, piece_mapping,
14    []) if reason is None: return candidate, total_elapsed,
15    False avoid_text, avoid_reason = candidate, reason return
16    piece_text, total_elapsed, True
```

Se il candidato viene rifiutato, il testo scartato e il motivo esatto del rifiuto diventano il contesto per il tentativo successivo, mentre la temperatura sale linearmente dalla base al massimo configurato (`temperature_for_attempt`). Se tutti i tentativi (5 di default) falliscono, la funzione non forza comunque un output: restituisce il testo originale non modificato, segnalando esplicitamente il ricorso a questo comportamento di ripiego. Il primo chunk di un record usa il template completo, con l'assegnazione di stile e la cronologia delle aperture di frase recenti da evitare; i chunk successivi usano un template più semplice *"a frammento"* (`as_fragment=True`), senza ripetere l'assegnazione di stile, per restare coerenti col registro già stabilito dal primo pezzo. Ogni chunk viene validato singolarmente, ma solo sull'integrità dei placeholder: il controllo di somiglianza, che richiede un confronto con varianti già accettate, si applica soltanto al termine, sul testo ricomposto (sezione 5.2.5).

5.2.5 Validazione: due funzioni per due modalità

La validazione di un candidato applica fino a tre criteri in sequenza all'interno di un'unica funzione (listato 5.20), più un quarto controllo esterno sulla ripetizione dell'apertura di frase.

Listing 5.20: La funzione di validazione condivisa da chunk e varianti complete, con soglia di somiglianza adattiva.

```
1 def validation_reason(candidate, mapping, accepted_so_far, max_similarity=None):
2     missing = [p for p in mapping if p not in candidate] if missing: return
3     f"you dropped the placeholder(s) {'', '.join(missing)} - every single one
4     must appear unchanged"
5
6     malformed = sorted(set(m for m in LOOSE_VAL_RE.findall(candidate) if m not
7     in mapping)) if malformed: return f"you wrote malformed or invented
8     placeholder-like text: {'', '.join(malformed)} - placeholders must be copied
9     exactly, never altered"
10
11     threshold = max_similarity if max_similarity is not None else
12     similarity_threshold_for(candidate) for prev in accepted_so_far: ratio =
13     difflib.SequenceMatcher(None, candidate.lower(), prev.lower()).ratio() if
14     ratio >= threshold: return f"this wording is too similar
15     ({ratio:.0%} match) to a variant already produced for this same request"
16     return None
```

Il controllo di somiglianza usa `difflib.SequenceMatcher`³ della libreria standard, con una soglia che dipende dalla lunghezza del testo: più permissiva (fino al 97%) per frasi brevi, più severa (85%) per frasi lunghe, altrimenti testi corti verrebbero quasi sempre scartati come duplicati per mancanza di margine di variazione lessicale. Poiché questa funzione riceve sempre una lista vuota quando è invocata sui singoli chunk, il controllo di somiglianza non si attiva mai a quel livello: è un ciclo su una lista vuota, non un controllo disattivato esplicitamente.

Per la modalità priva di mascheramento (sezione 5.2.8) esiste una seconda funzione di validazione, più permissiva sul primo controllo: un valore non è considerato perso se compare nella sua forma di suggerimento naturale, o come sinonimo plausibile case-insensitive, non solo nella sua forma letterale esatta, coerentemente col fatto che, in questa modalità, il modello vede il valore vero e ha il permesso esplicito di riformularlo.

³<https://docs.python.org/3/library/difflib.html>

5.2.6 Rotazione degli stili e memoria anti-ripetizione

La varietà stilistica tra le varianti di uno stesso record è governata da un oggetto che assegna gli stili in sequenza fissa e ricorda le generazioni recenti per stile (listato 5.21).

Listing 5.21: Assegnazione ciclica degli stili e memoria a finestra delle generazioni recenti.

```
1 class StyleTracker: def __init__(self, styles=None, recent_size=3,
2   max_chars=160, opener_window=12): self._styles = styles if styles is not
3   None else STYLES self._style_cycle = itertools.cycle(self._styles)
4   self._recent_texts = {tag: [] for tag, _ in self._styles}
5   self._opener_history = {tag: [] for tag, _ in self._styles}
6
7   def next_style(self): return next(self._style_cycle)
8
9   def remember(self, style_tag, text): sanitized = VAL_RE.sub("...", text)
10    preview = sanitized if len(sanitized) <= self._max_chars else
11    sanitized[:self._max_chars - 3] + "..." bucket =
12    self._recent_texts.setdefault(style_tag, []) bucket.append(preview) del
13    bucket[: -self._recent_size] openers =
14    self._opener_history.setdefault(style_tag, [])
15    openers.append(opener(text)) del openers[: -self._opener_window]
```

Gli stili (dieci registri predefiniti, da tecnico-database a colloquiale a statistico) vengono assegnati percorrendo un ciclo fisso (`itertools.cycle`⁴), non estratti a caso, per garantire una distribuzione uniforme su un numero elevato di record. `remember` viene chiamata solo su varianti accettate, mai su tentativi rifiutati: tiene una finestra delle ultime 3 formulazioni complete per stile (con i placeholder ancora presenti sostituiti da "...", per non confrontare mai codici opachi come se fossero testo naturale) e una finestra più ampia, le ultime 12, delle sole aperture di frase. La finestra qui è più ampia perché le prime due parole di una frase sono un'informazione più povera di una frase intera, e richiedono quindi uno storico più esteso per intercettare ripetizioni. Le due informazioni sono tenute in due dizionari distinti, invece che in un'unica struttura combinata, proprio perché servono controlli diversi con finestre di lunghezza diversa: tenerle separate evita di dover troncature artificialmente l'una alla lunghezza dell'altra.

⁴<https://docs.python.org/3/library/itertools.html#itertools.cycle>

5.2.7 Statistiche di qualità e resilienza

Ogni run accumula un insieme di contatori che permettono, a posteriori, di distinguere una variante uscita pulita da una uscita con un fallback, da una richiesta di variante che non ha prodotto nulla.

Listing 5.22: Calcolo della resa complessiva di un run, a partire dagli slot totali e da quelli completamente falliti.

```
1 def to_dict(self): slots_failed = len(self.failed_slots) yield_rate = (1 -  
2   (slots_failed / self.total_slots)) if self.total_slots else None return {  
3   "total_slots": self.total_slots, "slots_failed": slots_failed, "yield_rate":  
4   yield_rate, "fallback_count": self.fallback_count, "style_attempts":  
5   dict(self.style_attempts), "style_accepted": dict(self.style_accepted),  
6   "failed_slots": self.failed_slots, }
```

`fallback_count` conta le varianti uscite comunque, ma con almeno un chunk non riformulato; `failed_slots` registra invece le richieste di variante per cui, dopo aver esaurito i tentativi esterni di rigenerazione completa (non solo i tentativi per singolo chunk), non si è prodotto nulla; per ciascuna, viene conservato anche il dettaglio di ogni tentativo fallito e il relativo motivo. Le statistiche vengono salvate su disco dopo ogni record processato, non solo a fine run: un'esecuzione interrotta e ripresa continua ad accumulare sugli stessi contatori invece di ripartire da zero. Allo stesso scopo di resilienza, l'elenco dei `source_id` già presenti nel file di output viene ricostruito a ogni avvio, così che i record già completati vengano saltati in caso di ripresa dopo un'interruzione (*RNF2*).

5.2.8 La modalità senza mascheramento

Il sistema conserva, per ciascun record, anche un percorso di generazione alternativo che riproduce il comportamento pre-mascheramento: il testo intero viene sottoposto al modello con i valori veri elencati esplicitamente, senza alcuna suddivisione in chunk (listato 5.23).

Listing 5.23: La modalità senza mascheramento, mantenuta come termine di paragone esplicito e non come percorso di produzione alternativo.

```
1 def generate_variant_no_mask(model, text, values, style_tag, style_desc,  
2                               max_attempts, base_temp, max_temp, avoid_history,  
3                               slot_label, avoid_text=None, avoid_reason=None,
```

```
4         language="English", hints=None): # Deliberately no
5         chunking here: this mode exists to reproduce the #
6         pre-masking, single-shot behavior for direct
7         comparison, not to be a # second production path.
8         hints = hints or {} identity_mapping = {v: v for v
9         in values} value_list = ", ".join(f"{v}" for v
10        in values) ... reason =
11        validation_reason_with_hints(candidate, values,
12        hints, [])
```

Il commento nel codice sorgente è esplicito sul ruolo di questa modalità: non un secondo percorso di produzione, ma un termine di paragone diretto per quantificare in valutazione il beneficio del mascheramento (capitolo 6). L'evidenza raccolta in un run diagnostico su questa modalità conferma empiricamente la scelta progettuale discussa in capitolo 4: su 41 rifiuti registrati nel log di un run limitato a due record, 36 (87,8%) erano dovuti a valori letterali persi o alterati dal modello: un esito reso strutturalmente impossibile dal mascheramento, dato che in quella modalità il modello non ha mai accesso al valore reale durante la riformulazione.

Capitolo 6

Valutazione

6.1 Testing

Questa sezione riporta i problemi di qualità individuati nel dataset durante lo sviluppo, un confronto sperimentale tra due modalità di generazione, e le statistiche finali dei tre insiemi prodotti.

Problemi di qualità riscontrati

Nel corso dello sviluppo sono stati individuati cinque problemi di qualità veri e propri, ciascuno diagnosticato fino alla causa tecnica sottostante (figura 6.1). Una proposta di risoluzione per ciascuno di essi è discussa in capitolo 7. Due ulteriori comportamenti, inizialmente considerati problematici, sono stati in seguito riconosciuti come accettabili o addirittura desiderabili, e sono discussi separatamente al termine di questa sezione.

Un primo problema, isolato, ha riguardato la confusione tra l'acronimo clinico ECOG, relativo a una scala di stato funzionale oncologico, e il termine ecocardiogramma, un esame diagnostico distinto; il modello linguistico locale impiegato per la generazione dei sinonimi ha in un'occasione sostituito il primo con una forma testuale riconducibile al secondo. La causa è riconducibile ai limiti di conoscenza specifica del modello linguistico di dimensioni contenute impiegato per questo compito, che non dispone di una base affidabile di terminologia clinica e tende a

produrre un'associazione plausibile in base alla sola somiglianza superficiale tra i due termini, piuttosto che riconoscere di non avere l'informazione necessaria.

Un secondo problema, quantitativamente più rilevante e riconducibile alla medesima causa di fondo, ha riguardato l'allucinazione di codici di classificazione ICD-10. Il filtro di sicurezza del sistema di sinonimi, nella sua versione iniziale, escludeva dal tentativo di generazione del sinonimo i valori corrispondenti al pattern di un codice, riconosciuto tramite un prefisso testuale seguito dal carattere due punti; tale riconoscimento, tuttavia, era limitato a prefissi composti interamente da lettere, ed escludeva correttamente prefissi quali quello della nomenclatura SNOMED, ma non il prefisso della classificazione ICD-10, che contiene anche cifre. Come conseguenza, un medesimo codice ICD-10 è stato tradotto, in varianti diverse dello stesso record, in nomi di patologie non corrispondenti al codice originale e reciprocamente incoerenti tra loro, confermando che il modello linguistico non disponeva di una conoscenza affidabile della corrispondenza tra codice e patologia, ma produceva un'associazione plausibile priva di fondamento verificabile, lo stesso limite già osservato nel primo problema. Il problema ha interessato quarantasette record del dataset; il difetto specifico nel filtro di riconoscimento del pattern, che escludeva solo i prefissi alfabetici, è stato corretto.

Un terzo problema, riscontrato nella prima fase del lavoro di generazione manuale delle varianti per gli insiemi di validazione e di test, ha riguardato la produzione di testo non conforme a un registro di linguaggio naturale, quale la riproduzione di sintassi propria di un linguaggio di interrogazione strutturato, elenchi nella forma campo seguito da valore, e parentesi impiegate come dispositivo strutturale anziché come parte di un valore letterale originale. A differenza dei primi due problemi, la causa qui non è un limite del modello linguistico, ma la scrittura manuale dei modelli di frase impiegati in questa fase, che in alcuni casi rispecchiava abitudini di formulazione più vicine a un'interfaccia tecnica che al linguaggio naturale. Il problema è stato individuato tramite lettura diretta del testo prodotto e corretto riscrivendo i modelli di frase coinvolti.

Un quarto problema ha riguardato la concatenazione di due valori data adiacenti in un intervallo temporale, priva di un separatore testuale intermedio, risultante in una sequenza di cifre e trattini non interpretabile senza ambiguità da un lettore umano, poiché le date stesse contengono trattini come parte della propria format-

tazione. La causa è di natura puramente implementativa, non riconducibile ad alcun modello linguistico: il modello di frase impiegato per costruire il testo non prevedeva l'inserimento di una congiunzione quando due valori data si trovavano ad essere affiancati direttamente. Il problema, riscontrato in cinquantacinque occorrenze, è stato corretto inserendo sistematicamente una congiunzione testuale in tutti i punti in cui ciò si verificava.

Un quinto problema, distinto dai precedenti poiché non riconducibile a un difetto del sistema di generazione delle varianti, ha riguardato la presenza del valore letterale "nan" come unità di misura in un numero limitato di record, riscontrato in sette casi. Questo valore, tipico di una rappresentazione di dato mancante non gestita a monte, è stato preservato correttamente dal sistema di augmentation, che per costruzione non altera i valori letterali originali; la causa è quindi da ricondurre alla fase di generazione del dataset strutturato, non alla fase di augmentation linguistica.

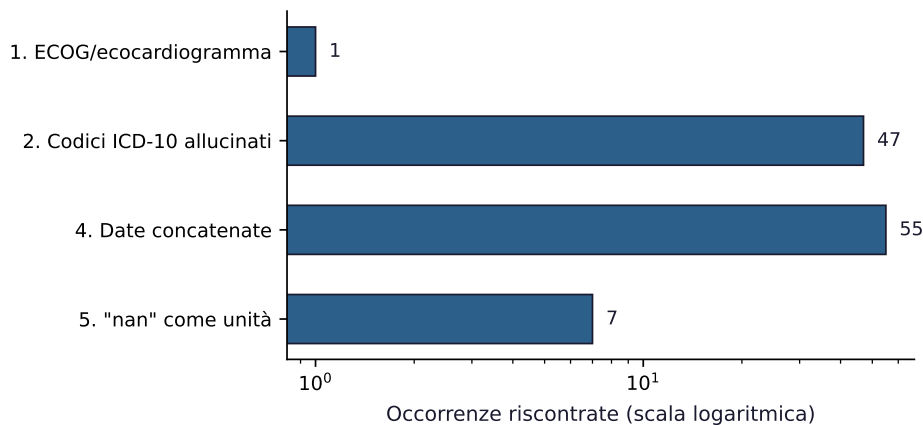


Figura 6.1: Occorrenze riscontrate per ciascuno dei quattro problemi di qualità quantificabili (il terzo problema, individuato tramite lettura diretta anziché conteggio sistematico, non compare in figura).

Due ulteriori comportamenti, inizialmente registrati come problemi durante lo sviluppo, meritano una lettura diversa. Il primo riguarda l'uso occasionale di una preposizione non canonica, "at" al posto di "on", in combinazione con valori riferiti a una linea di trattamento terapeutico, riscontrato in otto casi. Più che un difetto da correggere, questo comportamento può essere considerato una forma

di variazione naturale del linguaggio: un parlante reale non segue sempre con precisione le regole grammaticali, e una minima imprecisione di questo tipo può contribuire alla varietà stilistica del testo generato piuttosto che comprometterne la qualità.

Il secondo riguarda la presenza, in alcuni record, di due vincoli distinti sulla medesima proprietà scalare del paziente, quale la data di decesso, con valori incompatibili tra loro, risultante in una query logicamente priva di risultati possibili. Il fenomeno, ricondotto a una collisione tra due percorsi relazionali paralleli campionati indipendentemente durante la generazione, è risultato più frequente nel secondo passaggio di generazione, con duecentonovantacinque record su mille-trecentotrentuno esaminati in quella fase. Piuttosto che un difetto da correggere, questo comportamento riflette una condizione che si presenta anche nell'uso reale di un sistema di interrogazione: un utente può porre, per errore o per verifica, una domanda a cui nessun dato può corrispondere. Includere alcuni esempi di questo tipo nel dataset non compromette la qualità dell'addestramento, e i record interessati sono comunque stati esclusi dal dataset finale per mantenere la maggioranza dei casi orientata a query con una risposta effettiva.

Confronto tra generazione con e senza mascheramento

Al fine di verificare se l'assenza di mascheramento dei valori producesse una struttura linguistica delle varianti generate qualitativamente diversa rispetto alla modalità principale, è stato condotto un confronto sperimentale su un insieme di venti record di input, selezionati per rappresentare una gamma di complessità crescente, dal caso di un singolo valore vincolato fino al caso di dodici valori vincolati nella medesima query. Le due modalità sono state eseguite sul medesimo insieme di record, con i medesimi parametri di generazione.

Il numero complessivo di slot di generazione tentati è risultato identico tra le due modalità, sessantasei, poiché determinato dalla medesima formula adattiva basata sul numero di valori per record, indipendente dalla modalità di mascheramento. Il tasso di successo complessivo è risultato lievemente superiore nella modalità con mascheramento, novantotto virgola cinque percento contro novantasette percento nella modalità senza mascheramento. La causa dei fallimenti

si è rivelata inoltre di natura diversa tra le due modalità: nella modalità senza mascheramento, i fallimenti osservati sul medesimo record sono stati causati da una corrispondenza testuale identica al cento per cento tra tentativi di generazione distinti, condizione non osservata nella modalità con mascheramento, dove il fallimento osservato ha mostrato una corrispondenza testuale parziale, inferiore al novanta per cento.

Questo risultato suggerisce che l'assenza di segnaposti opachi, pur non compromettendo in modo sostanziale il tasso di successo complessivo, può ridurre la capacità del modello linguistico di produrre formulazioni distinte quando il testo con i valori visibili offre un numero limitato di modi naturali per esprimere la medesima richiesta.

Statistiche del dataset

Il dataset finale risulta suddiviso in tre insiemi distinti, destinati rispettivamente all'addestramento, alla validazione, e al test del modello. L'insieme di addestramento comprende 9976 varianti, prodotte a partire da 3443 record sorgente distinti, con una media di 2,90 varianti per record. Gli insiemi di validazione e di test comprendono rispettivamente 2022 e 2023 varianti, prodotte a partire da 428 record sorgente distinti ciascuno, con una media di 4,72 e 4,73 varianti per record, contro un obiettivo fisso di cinque varianti per record non sempre raggiunto per intero. Nessun record sorgente risulta condiviso tra i tre insiemi: la verifica sull'insieme dei relativi identificativi conferma una separazione completa, coerente con l'obiettivo di una valutazione di reale generalizzazione anziché di semplice memorizzazione.

La distribuzione delle varianti tra i dieci registri stilistici nell'insieme di addestramento risulta ragionevolmente uniforme, pur non perfettamente bilanciata (figura 6.2): lo stile più rappresentato, *technical_sql*, compare in 1237 varianti, contro le 569 dello stile meno rappresentato, *generational*, un rapporto di circa 2,2 a 1 tra il più e il meno frequente.

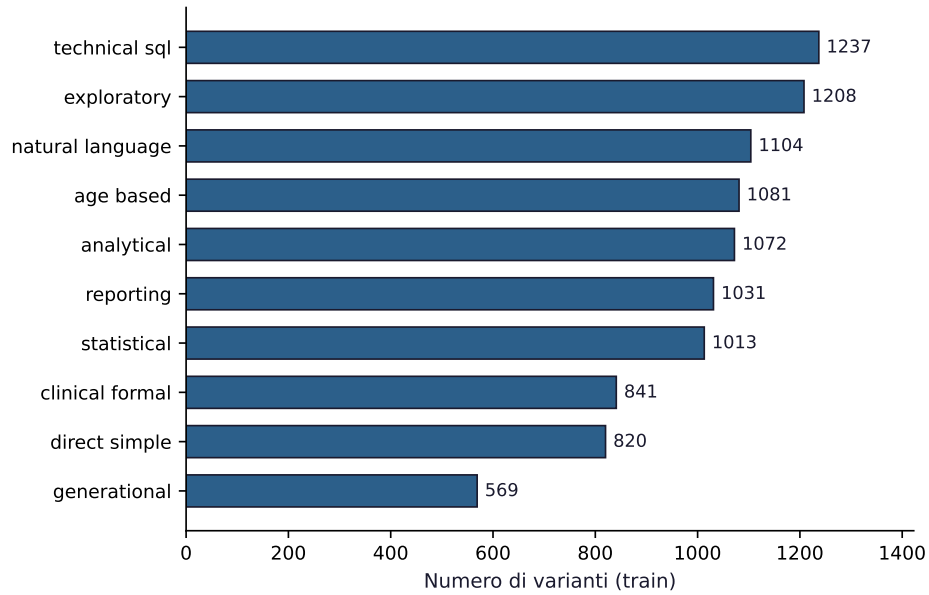


Figura 6.2: Distribuzione delle varianti tra i dieci stili linguistici, nell'insieme di addestramento.

Una differenza rilevante nella densità di parafrasi si osserva tra l'insieme di addestramento, prodotto tramite la formula adattiva basata sul numero di valori per record, e gli insiemi di validazione e di test, prodotti con un numero fisso di varianti per record: il primo presenta una media sensibilmente più bassa (2,90 contro le circa 4,7 degli altri due insiemi), una scelta deliberata per contenere il volume complessivo di addestramento senza sacrificare la varietà stilistica degli insiemi usati per la misurazione delle prestazioni.

Infine, la quota di varianti non riformulate rispetto al testo originale (*fallback*) risulta contenuta in tutti e tre gli insiemi: 0,58% nel train (58 varianti su 9 976), 0,35% sia in validation sia in test (7 varianti ciascuno).

6.2 Experimental evaluation / demos

Questa sezione riporta i risultati sperimentali del fine-tuning del modello linguistico sul dataset descritto nella sezione precedente, confrontando diverse configurazioni e analizzando la natura degli errori residui.

Metodologia

Il modello impiegato è Gemma 3 270M, adattato tramite LoRA (sezione 2.5.3) su un insieme di addestramento coerente con quello descritto in sezione 6.1. La metrica principale è l'*exact match*, la percentuale di query generate identiche carattere per carattere alla query di riferimento, calcolata su un insieme di test di cento esempi in stile coerente con l'insieme di addestramento (*trainstyle-test*); dove indicato, i risultati sono riportati anche su un secondo insieme di cento esempi fuori distribuzione (*claudetest*), formulato da un modello linguistico diverso da quello impiegato nella generazione del dataset stesso, per verificare la capacità di generalizzazione a formulazioni non viste in addestramento.

Selezione del checkpoint

Un primo addestramento di riferimento, condotto per 4 000 iterazioni, è stato seguito da un secondo addestramento più lungo, di 15 000 iterazioni. Anziché impiegare il checkpoint dell'ultima iterazione di quest'ultimo, la scelta è ricaduta sul checkpoint con la *validation loss* più bassa osservata durante l'addestramento, in corrispondenza dell'iterazione 7 000 (figura 6.3). Questa scelta metodologica si è rivelata determinante, portando l'*exact match* dal 15% del checkpoint finale (iterazione 15 000) al 46% del checkpoint scelto, sullo stesso modello e sugli stessi dati di addestramento.

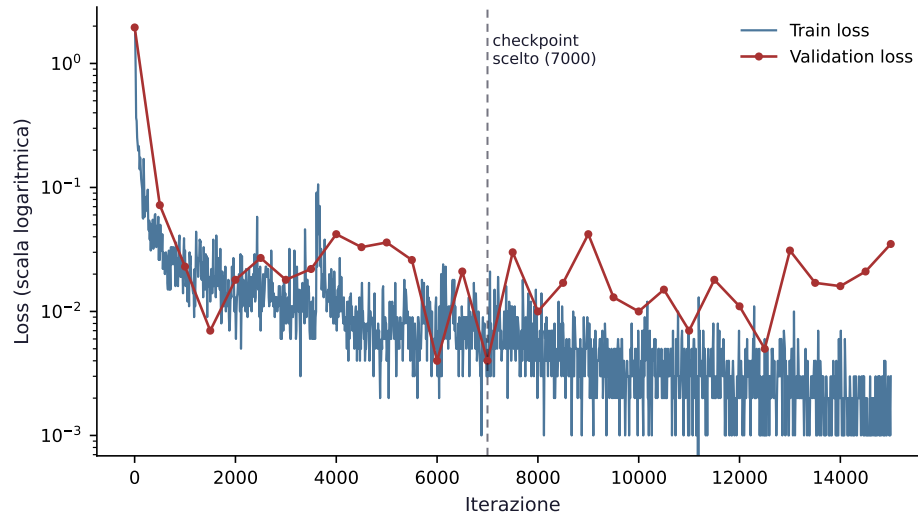


Figura 6.3: Andamento della train e della validation loss durante l’addestramento di 15 000 iterazioni. Il checkpoint scelto per la valutazione finale, in corrispondenza del minimo della validation loss, è evidenziato dalla linea tratteggiata.

Confronto tra configurazioni

Il checkpoint scelto (iterazione 7 000) raggiunge un exact match del **46%** sull’insieme di test in stile coerente con l’addestramento, un risultato solido per un modello di 270 milioni di parametri, addestrato con una tecnica efficiente come LoRA, su un compito che richiede di produrre query sintatticamente corrette e semanticamente vincolate a un’ontologia clinica reale (figura 6.4). Il primo addestramento, più breve, aveva raggiunto solo il 13% nella stessa condizione, confermando che l’esposizione prolungata ai dati, unita a una selezione accurata del checkpoint, produce un miglioramento sostanziale e non marginale. Un confronto diretto con il full fine-tuning, condotto sullo stesso modello di base per 6 000 iterazioni, mostra un exact match del 18% nella medesima condizione, sensibilmente inferiore al risultato ottenuto con LoRA a parità di ordine di grandezza di iterazioni.

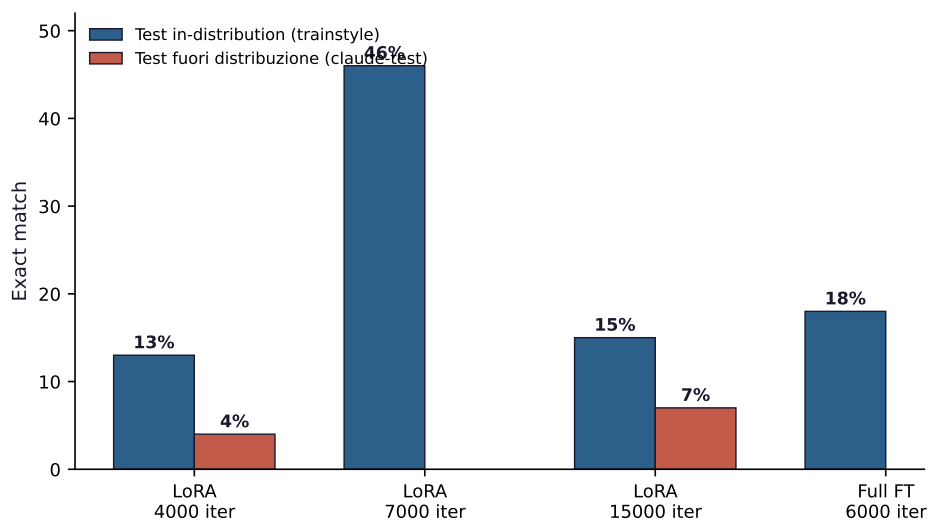


Figura 6.4: Confronto dell’exact match tra le configurazioni testate, sia sull’insieme di test in stile coerente con l’addestramento sia, dove disponibile, sull’insieme fuori distribuzione.

Sull’insieme di test fuori distribuzione, i risultati sono naturalmente più contenuti (4% per il primo addestramento, 7% per il checkpoint finale del secondo), un calo atteso trattandosi di formulazioni mai osservate durante l’addestramento e generate da un modello linguistico diverso da quello usato per produrre il dataset. Il checkpoint scelto (iterazione 7 000) non è stato valutato su questo insieme, essendo stato individuato successivamente come il più promettente sulla base della sola validation loss.

Analisi degli errori

Le cinquantaquattro righe non corrispondenti esattamente al checkpoint scelto sono state analizzate confrontando in modo automatico la struttura e i valori di ciascuna query generata con quella di riferimento (figura 6.5). Nella maggioranza dei casi, il 63%, la query generata utilizza un predicato, una classe, o un percorso relazionale diverso da quello corretto, tipicamente confondendo entità ontologiche simili tra loro (ad esempio `Patient` con una sua sottoclasse come `DrinkerPatient`, introdotta nella query senza che il prompt la richiedesse). Nel 24% dei casi la struttura della query è corretta, ma uno dei valori letterali richiesti differisce da

quello atteso. Nel restante 13% dei casi, la query è strutturalmente corretta e utilizza il predicato giusto, ma restituisce un identificativo o un codice errato al posto di quello corretto, ad esempio confondendo due codici della classificazione ICD-10 entrambi plausibili nel contesto ma diversi da quello richiesto.

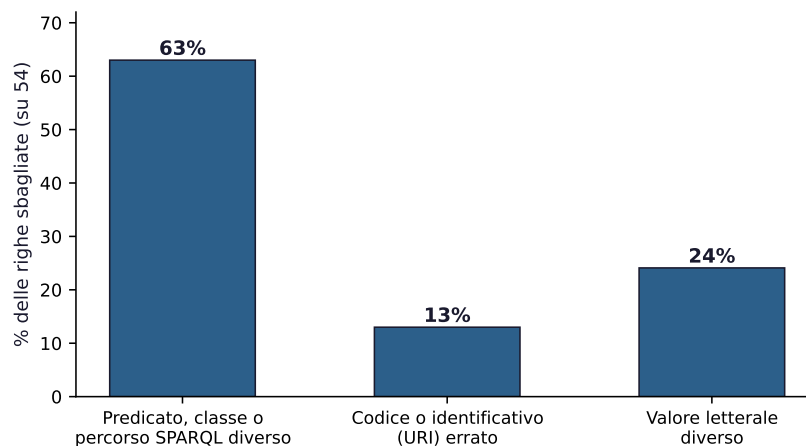


Figura 6.5: Distribuzione delle cinquantaquattro righe non corrispondenti per categoria di errore, sul checkpoint scelto (iterazione 7 000).

È significativo che, in nessuno dei casi analizzati, il modello abbia prodotto una query non sintatticamente valida o strutturalmente incoerente. Anche negli errori, il modello dimostra di aver appreso in modo affidabile la grammatica del linguaggio di interrogazione e la struttura generale delle query attese, concentrando le proprie difficoltà in un numero limitato di pattern riconoscibili e riconducibili a una causa comune, la discriminazione tra entità o valori simili tra loro, più che a un comportamento arbitrario o imprevedibile.

Capitolo 7

Conclusioni e sviluppi futuri

Questo lavoro ha affrontato il problema della mancanza di un dataset di addestramento per un modello linguistico capace di tradurre domande in linguaggio naturale in query SPARQL sul knowledge graph del Cancer Virtual Lab, progettando e implementando una pipeline automatica in due stadi per costruirlo, e verificandone l'efficacia tramite il fine-tuning concreto di un modello linguistico sui dati prodotti.

I quattro obiettivi specifici definiti in capitolo 3 possono essere considerati raggiunti. La copertura selettiva ed estendibilità (**O1**) è stata ottenuta tramite il meccanismo dei tier, che copre quarantacinque entità dello schema senza richiedere la copertura completa, ed è estendibile a nuove classi tramite una configurazione esterna che non richiede di modificare il codice del generatore (sezione 5.1.11). La varietà linguistica autentica (**O2**) è stata perseguita tramite lo stadio di augmentation, che produce dieci registri stilistici distinti a partire da ciascuna coppia generata, con un tasso di successo superiore al 97% anche nella modalità più restrittiva testata (sezione 6.1). La tracciabilità completa (**O3**) è garantita dall'identificativo di provenienza mantenuto per ogni variante, verificato non presentare alcuna sovrapposizione tra gli insiemi di addestramento, validazione e test (sezione 6.1). Il volume di dati adeguato al fine-tuning (**O4**) è confermato non solo dal numero complessivo di quattordicimilaventuno varianti prodotte, ma soprattutto dal fatto che il dataset si sia dimostrato concretamente utilizzabile per addestrare un modello con risultati misurabili, come riportato di seguito.

La valutazione sperimentale del modello ottenuto tramite fine-tuning ha prodotto un risultato solido, un exact match del 46% su un insieme di test in stile coerente con l'addestramento, ottenuto con LoRA su un modello di soli 270 milioni di parametri (sezione 6.2). Due risultati metodologici emersi durante la valutazione meritano di essere sottolineati. Il primo è che la scelta del checkpoint in base alla validation loss, piuttosto che all'ultima iterazione disponibile, ha prodotto un miglioramento determinante, dal 15% al 46% di exact match a parità di modello e di dati. Il secondo è che, a parità di ordine di grandezza di iterazioni, LoRA ha superato nettamente il full fine-tuning condotto come confronto (46% contro 18%). L'analisi degli errori residui ha inoltre mostrato che nessuno di essi consiste in una query sintatticamente non valida: il modello ha appreso in modo affidabile la grammatica del linguaggio di interrogazione, concentrando le proprie difficoltà nella discriminazione tra entità o valori simili tra loro, piuttosto che in un comportamento arbitrario.

Il processo di sviluppo ha inoltre permesso di individuare e diagnosticare cinque problemi di qualità genuini nel dataset generato (sezione 6.1), riconducendo ciascuno alla propria causa tecnica; due di essi sono già stati corretti nell'ambito di questo lavoro, mentre per i restanti tre è proposta una via di correzione nella sezione seguente. Due ulteriori comportamenti, inizialmente considerati problematici, sono stati infine riconosciuti come accettabili, un risultato che riflette una comprensione più matura, maturata nel corso del lavoro, di cosa costituisca realmente un difetto in un dataset di questo tipo rispetto a una variazione legittima.

7.1 Sviluppi futuri

7.1.1 Risoluzione dei problemi di qualità

Alcuni dei problemi di qualità discussi in sezione 6.1 non hanno ricevuto, nell'ambito di questo lavoro, una correzione alla radice, e rappresentano quindi possibili direzioni di sviluppo futuro.

Confusione tra acronimi clinici simili (sezione 6.1, p. 59). Una possibile correzione consisterebbe nel restringere la generazione di sinonimi per questa categoria di valori a un dizionario curato di equivalenze verificate, invece di affidarsi

alla generazione libera del modello linguistico locale, che non dispone di una base affidabile di terminologia clinica.

Allucinazione di codici ICD-10 (sezione 6.1, p. 60). Il difetto specifico nel filtro di riconoscimento è già stato corretto, ma una correzione più robusta consisterebbe nell'escludere del tutto dalla generazione libera qualunque valore riconducibile a un sistema di codifica controllato, sostituendola con una verifica rispetto alla tabella dei codici validi prima di accettare qualunque sostituzione proposta, così da eliminare il rischio alla radice invece di limitarsi a riconoscere meglio i pattern da escludere.

Valore letterale "nan" come unità di misura (sezione 6.1, p. 61). Una correzione consisterebbe nel riconoscere esplicitamente questo tipo di valore mancante durante la generazione del dataset strutturato, escludendo il record interessato o rappresentando l'assenza del dato in modo esplicito, invece di lasciare che un segnaposto tecnico non gestito raggiunga il testo finale.

7.1.2 Altre direzioni di sviluppo

Oltre alla risoluzione dei problemi di qualità appena discussi, il lavoro suggerisce alcune direzioni di sviluppo più ampie, emerse dai limiti riscontrati nel corso della valutazione più che da difetti puntuali.

Estensione della copertura dello schema. Il dataset attuale copre quarantacinque entità dello schema (O1), una scelta selettiva e non esaustiva. Il meccanismo di configurazione dei tier tramite file esterno (sezione 5.1.11) rende possibile estendere questa copertura ad altre classi e relazioni del knowledge graph senza modificare il codice del generatore; sfruttare concretamente questa possibilità per ampliare il dataset resta un lavoro da svolgere.

Valutazione a livello di esecuzione. La metrica di valutazione impiegata in questo lavoro, l'exact match, richiede una corrispondenza carattere per carattere tra la query generata e quella di riferimento (sezione 6.2). Una query semanticamente equivalente ma formulata in modo sintatticamente diverso, ad esempio con clausole in un ordine differente, viene quindi conteggiata come errore anche quando produrrebbe lo stesso risultato se eseguita sul knowledge graph. Una valutazione complementare, che esegua le query generate sull'endpoint di CVL e ne confronti

i risultati restituiti anziché il solo testo, fornirebbe una stima più accurata delle prestazioni reali del modello.

Altre tecniche o modelli di adattamento. Questo lavoro ha confrontato LoRA con il full fine-tuning (sezione 6.2), ma non ha testato QLoRA, pure introdotta in sezione 2.5.3 come tecnica in grado di ridurre ulteriormente l’occupazione di memoria rispetto a LoRA; allo stesso modo, non sono stati testati modelli di base diversi da Gemma 3 270M, di dimensioni maggiori o architettura differente, che potrebbero offrire una migliore capacità di generalizzazione a scapito di un costo computazionale più elevato.

Resilienza a interruzioni estesa al generatore. Il requisito di resilienza a interruzioni (RNF2) è soddisfatto dallo stadio di augmentation, che ricostruisce a ogni avvio l’elenco dei record già processati, ma non dal generatore, che non conserva alcuna informazione tra un’esecuzione interrotta e la successiva. Estendere un meccanismo analogo anche al generatore ridurrebbe il lavoro perso in caso di interruzione durante le fasi iniziali della pipeline.

Bibliografia

- [AAG⁺25] P. De Angelis, A. Andalò, N. Gentili, L. Giorgetti, L. Ridolfi, R. Pasolini, A. Pagliarani, M. Cavallucci, R. Vespignani, and Antonella Carbonaro. Cancer virtual lab: una piattaforma sicura e interoperabile basata su knowledge graph e large language model per la ricerca oncologica. *Recenti Progressi in Medicina*, 116(10):601–602, 2025.
- [CGR⁺25] Antonella Carbonaro, L. Giorgetti, L. Ridolfi, R. Pasolini, A. Pagliarani, M. Cavallucci, A. Andalò, L. Del Gaudio, P. De Angelis, R. Vespignani, and N. Gentili. From raw data to research-ready: A FHIR-based transformation pipeline in a real-world oncology setting. *Computers in Biology and Medicine*, 197:111051, 2025.
- [DBAL19] Mohnish Dubey, Debayan Banerjee, Abdelrahman Abdelkawi, and Jens Lehmann. Lc-quad 2.0: A large dataset for complex question answering over wikidata and dbpedia. In *The Semantic Web – ISWC 2019*, pages 69–78. Springer, 2019.
- [HSW⁺22] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [LdCF25] A. B. Labate, B. W. S. R. de Carvalho, and S. R. Fiorini. Synthetic dataset generation and validation with LLMs for text2sparql in Wikidata graphs. In *Proceedings of the Wikidata Workshop 2025*, 2025.

- [SCKS20] B. A. Spiegel, V. Cheong, J. E. Kaplan, and A. Sanchez. MK-SQuIT: Synthesizing questions using iterative template-filling, 2020.
- [TMDL17] Priyansh Trivedi, Gaurav Maheshwari, Mohnish Dubey, and Jens Lehmann. Lc-quad: A corpus for complex question answering over knowledge graphs. In *The Semantic Web – ISWC 2017*, pages 210–218. Springer, 2017.

Acknowledgements

la vuoi